Library

Parameters

`BifurcationKit.NewtonPar` — Type

```
struct NewtonPar{T, L<:BifurcationKit.AbstractLinearSolver, E<:AbstractEigenSolver}
```

Returns a variable containing parameters to affect the `newton` algorithm when solving `F(x) = 0`.

Arguments for line search (Armijo)

- `linesearch = false`: use line search algorithm (i.e. Newton with Armijo's rule)
- `α = 1.0`: initial value of α (damping) parameter for line search algorithm
- `αmin = 0.001`: minimal value of the damping `alpha`

Mutating

For performance reasons, we decided to use an immutable structure to hold the parameters. One can use the package `Accessors.jl` to drastically simplify the mutation of different fields. See the tutorials for examples.

Internal fields

- `tol::Any`: absolute tolerance for `F(x)` Default: 1.0e-12

- `linsolver::BifurcationKit.AbstractLinearSolver`: linear solver, must be `<: AbstractLinearSolver` Default: DefaultLS()
- `eigsolver::AbstractEigenSolver`: eigen solver, must be `<: AbstractEigenSolver` Default: DefaultEig()
- `linesearch::Bool`: Default: false
- `α::Any`: Default: convert(typeof(tol), 1.0)
- `αmin::Any`: Default: convert(typeof(tol), 0.001)

`BifurcationKit.ContinuationPar` — Type

```
struct ContinuationPar{T, S<:BifurcationKit.AbstractLinearSolver, E<:AbstractEigenSolver}
```

Returns a variable containing the parameters to affect the `continuation` algorithm used to solve `F(x, p) = 0`.

Arguments

- `dsmin, dsmax` are the minimum, maximum allowed arc-length value. It controls the density of points in the computed branch of solutions.
- `ds = 0.01` is the initial arc-length.
- `p_min, p_max` allowed parameter range for `p`
- `max_steps = 100` maximum number of continuation steps
- `newton_options::NewtonPar`: options for the Newton algorithm
- `save_to_file = false`: save to file. A name is automatically generated or can be defined in `continuation`. This requires `using JLD2`.
- `save_sol_every_step::Int64 = 0` at which continuation steps do we save the current solution
- `plot_every_step = 10` at which continuation steps do we plot the current solution

Handling eigen-elements, their computation is triggered by the argument `detect_bifurcation` (see below)

- `nev = 3` number of eigenvalues to be computed. It is automatically increased to have at least `nev` unstable eigenvalues. To be set for proper bifurcation detection. See Detection of bifurcation points of Equilibria for more information.
- `save_eig_every_step = 1` record eigen vectors every specified steps. **Important** for memory limited resource, *e.g.* GPU.
- `save_eigenvectors = true` **Important** for memory limited resource, *e.g.* GPU.

Handling bifurcation detection

- `tol_stability = 1e-10` lower bound on the real part of the eigenvalues to test for stability of equilibria and periodic orbits
- `detect_fold = true` detect Fold bifurcations? It is a useful option although the detection of Fold is cheap. Indeed, it may happen that there is a lot of Fold points and this can saturate the memory in memory limited devices (e.g. on GPU)
- `detect_bifurcation::Int` ∈ {0, 1, 2, 3} If set to 0, nothing is done. If set to 1, the eigen-elements are computed. If set to 2, the bifurcations points are detected during the continuation run, but not located precisely. If set to 3, a bisection algorithm is used to locate the bifurcations points (slower). The possibility to switch off detection is a useful option. Indeed, it may happen that there are a lot of bifurcation points and this can saturate the memory of memory limited devices (e.g. on GPU)
- `dsmin_bisection = 1e-16` minimal `ds` for the bisection algorithm for locating bifurcation points
- `n_inversion = 2` number of sign inversions in bisection algorithm
- `max_bisection_steps = 15` maximum number of bisection steps
- `tol_bisection_eigenvalue = 1e-16` tolerance on real part of eigenvalue to detect bifurcation points in the bisection steps

Handling `ds` adaptation (see `continuation` for more information)

Handling event detection

GitHub

- `detect_event::Int` ∈ {0, 1, 2} If set to 0, nothing is done. If set to 1, the event locations are sought during the continuation run, but not located precisely. If set to 2, a bisection algorithm is used to locate the event (slower).
- `tol_param_bisection_event = 1e-16` tolerance on parameter to locate event

Misc

- `η = 150.` parameter to estimate tangent at first point with parameter p_0 + ds / η
- `detect_loop` [WORK IN PROGRESS] detect loops in the branch and stop the continuation

Mutating

For performance reasons, we decided to use an immutable structure to hold the parameters. One can use the package `Accessors.jl` to drastically simplify the mutation of different fields. See tutorials for more examples.

BifurcationKit.cbMaxNorm — Type

```
cb = cbMaxNorm(maxres)
```

Create a callback used to reject residuals larger than `cb.maxres` in the Newton iterations. See docs for `newton`.

BifurcationKit.cbMaxNormAndΔp — Type

```
cb = cbMaxNormAndΔp(maxres, δp)
```

Create a callback used to reject residuals larger than `cb.maxres` or parameter step larger than `δp` in the Newton iterations. See docs for `newton`.

Results

BifurcationKit.NonLinearSolution — Type

Structure which holds the solution from application of Newton-Krylov algorithm to a nonlinear problem.

For example

```
sol = newton(prob, NewtonPar())
```

methods

- `converged(sol)` return whether the solution has converged.

Internal fields

- `u::Any`: solution
- `prob::Any`: nonlinear problem, typically a `BifurcationProblem`
- `residuals::Any`: sequence of residuals
- `converged::Bool`: has algorithm converged?
- `itnewton::Int64`: number of newton steps
- `itlineartot::Any`: total number of linear iterations

BifurcationKit.ContResult — Type

GitHub

Structure which holds the results after a call to `continuation`.

You can see the propertynames of a result `br` by using `propertynames(br)` or `propertynames(br.branch)`.

Internal fields

- `branch::StructArrays.StructArray`: holds the low-dimensional information about the branch. More precisely, `branch[i+1]` contains the following information `(record_from_solution(u, param), param, itnewton, itlinear, ds, 0, n_unstable, n_imag, stable, step)` for each continuation step `i`.
  - `itnewton` number of Newton iterations.
  - `itlinear` total number of linear iterations during newton (corrector).
  - `n_unstable` number of eigenvalues with positive real part for each continuation step (to detect stationary bifurcation).
  - `n_imag` number of eigenvalues with positive real part and non zero imaginary part at current continuation step (useful to detect Hopf bifurcation).
  - `stable` stability of the computed solution for each continuation step. Hence, `stable` should match `eig[step]` which corresponds to `branch[k]` for a given `k`.
  - `step` continuation step (here equal `i`).
- `eig::Array{@NamedTuple{eigenvals::Teigvals, eigenvecs::Teigvec, converged::Bool, step::Int64}, 1} where {Teigvals, Teigvec}`: A vector with eigen-elements at each continuation step.
- `sol::Any`: Vector of solutions sampled along the branch. This is set by the argument `save_sol_every_step::Int64` (default 0) in `ContinuationPar`.
- `contparams::Any`: The parameters used for the call to `continuation` which produced this branch. Must be a `ContinuationPar`.
- `kind::BifurcationKit.AbstractContinuationKind`: Type of solutions computed in this branch. Default: EquilibriumCont()
- `prob::Any`: Bifurcation problem used to compute the branch, useful for branch switching. For example, when computing periodic orbits, the functional `Trapeze`, `Shooting`... will be saved here. Default: nothing
- `specialpoint::Vector`: A vector holding the list of detected bifurcation points. See `SpecialPoint` for a list of special points.
- `alg::Any`: Continuation algorithm used for the computation of the branch

Associated methods

- `length(br)` number of the continuation steps.
- `show(br)` display information about the branch.
- `propertynames(br)` give the propertynames of a result.
- `eigenvals(br, ind)` returns the eigenvalues for the ind-th continuation step.
- `eigenvec(br, ind, indev)` returns the indev-th eigenvector for the ind-th continuation step.
- `get_normal_form(br, ind)` compute the normal form of the ind-th points in `br.specialpoint`.
- `getlens(br)` return the parameter axis used for the branch.
- `getlenses(br)` return the parameter two axis used for the branch when 2 parameters continuation is used (Fold, Hopf, NS, PD).
- `get_solx(br, k)` returns the k-th solution on the branch.
- `get_solp(br, k)` returns the parameter value associated with k-th solution on the branch.
- `getparams(br)` Parameters passed to continuation and used in the equation `F(x, par) = 0`.
- `getparams(br, ind)` Parameters passed to continuation and used in the equation `F(x, par) = 0` for the ind-th continuation step.
- `setparam(br, p0)` set the parameter value `p0` according to `::Lens` for the parameters of the problem `getprob(br)`.
- `getlens(br)` get the lens used for the computation of the branch.
- `eigenvals(br, ind)` give the eigenvalues at continuation step `ind`.

- `type(br, ind)` returns the type of the ind-th bifurcation point.

GitHub

- `br[k+1]` gives information about the k-th step. A typical run yields the following.
- `get_solution(br, ind)` returns the ind-th solution.

```
julia> br[1]
(x = 0.0, param = 0.1, itnewton = 0, itlinear = 0, ds = -0.01, θ = 0.5, n_unstable = 2, n_imag = 2, s
```

which provides the value `param` of the parameter of the current point, its stability, information on the newton iterations, etc. The fields can be retrieved using `propertynames(br.branch)`. This information is stored in `br.branch` which is a `StructArray`. You can thus extract the vector of parameters along the branch as

```
julia> br.param
10-element Vector{Float64}:
 0.1
 0.08585786437626905
 0.06464466094067263
 0.03282485578727799
-1.2623798512809007e-5
-0.07160718539365075
-0.17899902778635765
-0.3204203840236672
-0.4618417402609767
-0.5
```

- `continuation(br, ind)` performs automatic branch switching (aBS) from ind-th bifurcation point. Typically branching from equilibrium to equilibrium, or periodic orbit to periodic orbit.
- `continuation(br, ind, lens2)` performs two parameters `(getlens(br), lens2)` continuation of the ind-th bifurcation point.
- `continuation(br, ind, probPO::AbstractBoundaryValueDiscretization)` performs aBS from ind-th bifurcation point (which must be a Hopf bifurcation point) to branch of periodic orbits.

BifurcationKit.Branch — Type

```
struct Branch{Tkind, Tprob, T<:Union{ContResult, Vector{<:ContResult}}, Tbp} <: BifurcationKit.Abstra
```

A Branch is a structure which encapsulates the result of the computation of a branch bifurcating from a bifurcation point.

- `γ::Union{ContResult, Vector{<:ContResult}}`: Set of branches branching off the bifurcation point `bp`.
- `bp::Any`: Bifurcation point. It is thought as the root of the branches in γ.

BifurcationKit.SpecialPoint — Type

```
struct SpecialPoint{T, Tp, Tv, Tvτ} <: BifurcationKit.AbstractBifurcationPoint
```

Structure to record special points on a curve. There are two types of special points that are recorded in this structure: bifurcation points and events (see https://bifurcationkit.github.io/BifurcationKitDocs.jl/dev/EventCallback/).

Internal fields

- `type::Symbol`: Description of the special points. In case of `Events`, this field records the user passed name to the event, or the default `:userD`, `:userC`. In case of bifurcation points, it can be one of the following names:

```
- :bp Bifurcation point, simple eigenvalue crossing the imaginary axis
- :fold Fold point
```

```
  - :gh Generalized Hopf point (also called Bautin point)
  - :bt Bogdanov-Takens point
  - :zh Zero-Hopf point
  - :hh Hopf-Hopf point
  - :ns Neimark-Sacker point
  - :pd Period-doubling point
  - :R1 Strong resonance 1:1 of periodic orbits
  - :R2 Strong resonance 1:2 of periodic orbits
  - :R3 Strong resonance 1:3 of periodic orbits
  - :R4 Strong resonance 1:4 of periodic orbits
  - :foldFlip Fold / Flip of periodic orbits
  - :foldNS Fold / Neimark-Sacker of periodic orbits
  - :pdNS  Period-Doubling / Neimark-Sacker of periodic orbits
  - :gpd Generalized Period-Doubling of periodic orbits
  - :nsns Double Neimark-Sacker of periodic orbits
  - :ch Chenciner bifurcation of periodic orbits
 Default: :none
```

- `idx::Int64`: Index in `br.branch` or `br.eig` (see `ContResult`) for which the bifurcation occurs. Default: 0
- `param::Any`: Parameter value at the special point (this is an estimate). Default: 0.0
- `norm::Any`: Norm of the equilibrium at the special point. Default: 0.0
- `printsol::Any`: `printsol = record_from_solution(x, param)` where `record_from_solution` is one of the arguments to `continuation`. Default: 0.0
- `x::Any`: Solution at the special point. Default: Vector{T}(undef, 0)
- `τ::BorderedArray{Tvτ, T} where {T, Tvτ}`: Tangent along the branch at the special point. Default: BorderedArray(x, zero(T))
- `ind_ev::Int64`: Eigenvalue index responsible for detecting the special point (if applicable). Default: 0
- `step::Int64`: Continuation step at which the special occurs. Default: 0
- `status::Symbol`: `status ∈ {:converged, :guess, :guessL}` indicates whether the bisection algorithm was successful in detecting the special (bifurcation) point. If `status == :guess`, the bisection algorithm failed to meet the requirements given in `::ContinuationPar`. Same for `status == :guessL` but the bisection algorithm stopped on the left of the bifurcation point. Default: :guess
- `δ::Tuple{Int64, Int64}`: `δ = (δr, δi)` where δr indicates the change in the number of unstable eigenvalues and δi indicates the change in the number of unstable eigenvalues with nonzero imaginary part. `abs(δr)` is thus an estimate of the dimension of the kernel of the Jacobian at the special (bifurcation) point. Default: (0, 0)
- `precision::Any`: Precision in the location of the special point Default: -1
- `interval::Tuple{T, T} where T`: Interval parameter containing the special point Default: (0, 0)

Associated methods

- `BifurcationKit.type(::SpecialPoint)` returns the bifurcation type (`::Symbol`)

`BifurcationKit.BifDiagNode` — Type

Structure to hold a connected component of a bifurcation diagram which is encoded as a tree of `BifDiagNode`(s).

Internal fields

- `level::Int64`: current recursion level in the tree.
- `code::Int64`: code for finding the current node in the tree, this is the index of the bifurcation point from which γ branches off.
- `γ::Any`: branch associated to the current node.
- `child::Any`: children of current node. These are the different branches off the bifurcation point in γ.

Methods

- `from(diagram)` return the parent bifurcation point.

GitHub

- `diagram[code]` For example `diagram[1,2,3]` returns `diagram.child[1].child[2].child[3]`. This is essentially the Ulam-Harris-Neveu labeling.

Problems

`BifurcationKit.BifFunction` — Type

```
struct BifFunction{Tf, TFinp, Tdf, Tdfad, Tj, Tjad, TJinp, Td2f, Td2fc, Td3f, Td3fc, Tδ, Tjet} <: Bif
```

Structure to hold the vector field and its derivatives. It should rarely be called directly. Also, in essence, it is very close to `SciMLBase.ODEFunction`.

Internal fields

- `F::Any`: Vector field. Function of type out-of-place `result = f(x, p)` or inplace `f(result, x, p)`. For type stability, the types of `x` and `result` should match
- `F!::Any`: Same as F but inplace with signature F!(result, x, p)
- `dF::Any`: Differential of `F` with respect to `x`, signature `dF(x,p,dx)`
- `dFad::Any`: Adjoint of the Differential of `F` with respect to `x`, signature `dFad(x,p,dx)`
- `J::Any`: Jacobian of `F` at `(x, p)`. It can assume three forms. 1. Either `J` is a function and `J(x, p)` returns a `::AbstractMatrix`. In this case, the default arguments of `contparams::ContinuationPar` will make `continuation` work. 2. Or `J` is a function and `J(x, p)` returns a function taking one argument `dx` and returning `dr` of the same type as `dx`. In our notation, `dr = J * dx`. In this case, the default parameters of `contparams::ContinuationPar` will not work and you have to use a Matrix Free linear solver, for example `GMRESIterativeSolvers`, 3. Or `J` is a function and `J(x, p)` returns a variable `j` which can assume any type. Then, you must implement a linear solver `ls` as a composite type, subtype of `AbstractLinearSolver` which is called like `ls(j, rhs)` and which returns the solution of the jacobian linear system. See for example `examples/SH2d-fronts-cuda.jl`. This linear solver is passed to `NewtonPar(linsolver = ls)` which itself passed to `ContinuationPar`. Similarly, you have to implement an eigensolver `eig` as a composite type, subtype of `AbstractEigenSolver`.
- `Jᵗ::Any`: jacobian adjoint, it should be implemented in an efficient manner. For matrix-free methods, `transpose` is not readily available and the user must provide a dedicated method. In the case of sparse based jacobian, `Jᵗ` should not be passed as it is computed internally more efficiently, i.e. it avoids recomputing the jacobian as it would be if you pass `Jᵗ = (x, p) -> transpose(dF(x, p))`.
- `J!::Any`: Inplace jacobian
- `d2F::Any`: Second Differential of `F` with respect to `x`, signature `d2F(x,p,dx1,dx2)`
- `d3F::Any`: Third Differential of `F` with respect to `x`, signature `d3F(x,p,dx1,dx2,dx3)`
- `d2Fc::Any`: [internal] Second Differential of `F` with respect to `x` which accept complex vectors dxi
- `d3Fc::Any`: [internal] Third Differential of `F` with respect to `x` which accept complex vectors dxi
- `isSymmetric::Bool`: Whether the jacobian is auto-adjoint.
- `δ::Any`: used internally to compute derivatives (with finite differences), for example for normal form computation and codim 2 continuation.
- `inplace::Bool`: optionally sets whether the function is inplace or not. You can use `in_bisection(state)` to inquire whether the current state is in bisection mode.
- `jet::Any`: jet of the vector field

Methods

- `residual(pb::BifFunction, x, p)` calls `pb.F(x,p)`
- `residual!(pb::BifFunction, o, x, p)` calls `pb.F(o, x, p)`
- `jacobian(pb::BifFunction, x, p)` calls `pb.J(x, p)`
- `dF(pb::BifFunction, x, p, dx)` calls `pb.dF(x, p, dx)`

- etc

GitHub

BifurcationKit.BifurcationProblem — Type

```
struct BifurcationProblem{Tvf, Tu, Tp, Tl<:Union{typeof(identity), IndexLens, PropertyLens, ComposedF
```

Structure to hold a bifurcation problem. Generic case, the user has to set most options.

Methods

- `re_make(pb; kwargs...)` modify a bifurcation problem
- `getu0(pb)` calls `pb.u0`
- `getparams(pb)` calls `pb.params`
- `getlens(pb)` calls `pb.lens`
- `getparam(pb)` calls `get(pb.params, pb.lens)`
- `setparam(pb, p0)` calls `set(pb.params, pb.lens, p0)`
- `record_from_solution(pb)` calls `pb.recordFromSolution`
- `plot_solution(pb)` calls `pb.plotSolution`
- `is_symmetric(pb)` calls `is_symmetric(pb.prob)`
- `getdelta(prob)`

Constructors

- `BifurcationProblem(F, u0, params, lens)` all derivatives are computed using ForwardDiff.
- `BifurcationProblem(F, u0, params, lens; J, Jᵗ, d2F, d3F, kwargs...)` and `kwargs` are the fields above. You can pass your own jacobian with `J` (see `BifFunction` for description of the jacobian function) and jacobian adjoint with `Jᵗ`. For example, this can be used to provide finite differences based jacobian using `BifurcationKit.finite_differences`. You can also pass
  - `record_from_solution` see above
  - `plot_solution` see above
  - `issymmetric[=false]` whether the jacobian is symmetric, this remove the need of providing an adjoint
  - `jvp` jacobian-vector product, signature `jvp(x, p, dx)`
  - `vjp` vector-jacobian product (adjoint of jvp), signature `vjp(x, p,dx)`
  - `d2F` second Differential of `F` with respect to `x`, signature `d2F(x, p, dx1, dx2)`
  - `d3F` third Differential of `F` with respect to `x`, signature `d3F(x, p, dx1, dx2, dx3)`
  - `save_solution` specify a particular way to record solution which are written in `br.sol`. This can be useful in very particular situations and we recommend using `record_from_solution` instead. For example, it is used internally to record the mesh in the collocation method because this mesh can be modified.

Internal fields

- `VF::Any`: Vector field, typically a `BifFunction`.
- `u0::Any`: Initial guess.
- `params::Any`: Parameters.
- `lens::Union{typeof(identity), IndexLens, PropertyLens, ComposedFunction}`: Typically a `Accessors.PropertyLens`. It specifies which parameter axis among `params` is used for continuation. For example, if `par = (α = 1.0, β = 1.78)`, we can perform continuation w.r.t. `α` by using `lens = (@optic _.α)`. If you have an array `par = [1.0, 2.0]` and want to perform continuation w.r.t. the first variable, you can use `lens = (@optic _[1])` or pass directly `lens = 1`. For more information, we refer to `Accessors.jl`.
- `plotSolution::Any`: user function to plot solutions during continuation. Signature: `plot_solution(x, p; kwargs...)` for `Plot.jl` and `plot_solution(ax, x, p; ax1 = nothing, kwargs...)` for the `Makie.jl`.

vectors is not possible for memory reasons (for example on GPU). This function can return pretty much everything but you should keep it small. For example, you can do `(x, p; k...) -> (x1 = x[1], x2 = x[2], nrm = norm(x))` or simply `(x, p; k...) -> (sum(x), 1)` or `(x, p; k...) -> [x[1], x[2]]`. This will be stored in `contres.branch` where `contres::AbstractBranchResult` is the continuation curve of the bifurcation problem. Finally, the first component is used for plotting in the continuation curve.

- `save_solution::Any`: Function to save the full solution on the branch. Some problem are updated during computation (like periodic orbit functional with adaptive mesh) and this function allows to save the state of the problem along with the solution itself. Note that this should allocate the output (i.e. not as a view). Signature: `save_solution(x, p)`. Defaults to `save_solution_default(x, p) = x`.
- `update!::Any`: Function to update the problem after each continuation step. Defaults to `update_default`. It has signature `update!(prob, iter, state)` where `prob` is the current bifurcation problem, `iter::ContIterable` the current iterable and `state::ContState` the current state of the continuation. It should return `true` if the update was successful and `false` otherwise. The continuation will stop if `false` is returned. This type of function is useful for example to update the problem internals like meshes, preconditioners, etc.

`BifurcationKit.ODEBifProblem` — Type

```
struct ODEBifProblem{Tvf, Tu, Tp, Tl<:Union{typeof(identity), IndexLens, PropertyLens, ComposedFuncti
```

Structure to hold a bifurcation problem. Specific to Ordinary Differential Equations (ODE). The options are set accordingly. 🚧🚧 This is work in progress 🚧🚧.

Methods

- `re_make(pb; kwargs...)` modify a bifurcation problem
- `getu0(pb)` calls `pb.u0`
- `getparams(pb)` calls `pb.params`
- `getlens(pb)` calls `pb.lens`
- `getparam(pb)` calls `get(pb.params, pb.lens)`
- `setparam(pb, p0)` calls `set(pb.params, pb.lens, p0)`
- `record_from_solution(pb)` calls `pb.recordFromSolution`
- `plot_solution(pb)` calls `pb.plotSolution`
- `is_symmetric(pb)` calls `is_symmetric(pb.prob)`
- `getdelta(prob)`

Constructors

- `ODEBifProblem(F, u0, params, lens)` all derivatives are computed using ForwardDiff.
- `ODEBifProblem(F, u0, params, lens; J, Jᵗ, d2F, d3F, kwargs...)` and `kwargs` are the fields above. You can pass your own jacobian with `J` (see `BifFunction` for description of the jacobian function) and jacobian adjoint with `Jᵗ`. For example, this can be used to provide finite differences based jacobian using `BifurcationKit.finite_differences`. You can also pass
  - `record_from_solution` see above
  - `plot_solution` see above
  - `issymmetric[=false]` whether the jacobian is symmetric, this remove the need of providing an adjoint
  - `jvp` jacobian-vector product, signature `jvp(x, p, dx)`
  - `vjp` vector-jacobian product (adjoint of jvp), signature `vjp(x, p,dx)`
  - `d2F` second Differential of `F` with respect to `x`, signature `d2F(x, p, dx1, dx2)`
  - `d3F` third Differential of `F` with respect to `x`, signature `d3F(x, p, dx1, dx2, dx3)`
  - `save_solution` specify a particular way to record solution which are written in `br.sol`. This can be useful in very particular situations and we recommend using `record_from_solution` instead. For example, it is used internally to record the mesh in the collocation method because this mesh can be modified.

- `F::Any`: vector field, typically a `BifFunction`.
- `u0::Any`: Initial guess.
- `params::Any`: Parameters.
- `lens::Union{typeof(identity), IndexLens, PropertyLens, ComposedFunction}`: Typically a `Accessors.PropertyLens`. It specifies which parameter axis among `params` is used for continuation. For example, if `par = (α = 1.0, β = 1.78)`, we can perform continuation w.r.t. `α` by using `lens = (@optic _.α)`. If you have an array `par = [1.0, 2.0]` and want to perform continuation w.r.t. the first variable, you can use `lens = (@optic _[1])` or pass directly `lens = 1`. For more information, we refer to `Accessors.jl`.
- `plotSolution::Any`: user function to plot solutions during continuation. Signature: `plot_solution(x, p; kwargs...)` for `Plot.jl` and `plot_solution(ax, x, p; ax1 = nothing, kwargs...)` for the `Makie.jl`.
- `recordFromSolution::Any`: `record_from_solution = (x, p; k...) -> norm(x)` function used to record a few indicators about the solution. It could be `norm` or `(x, p; k...) -> x[1]`. This is also useful when saving several huge vectors is not possible for memory reasons (for example on GPU). This function can return pretty much everything but you should keep it small. For example, you can do `(x, p; k...) -> (x1 = x[1], x2 = x[2], nrm = norm(x))` or simply `(x, p; k...) -> (sum(x), 1)` or `(x, p; k...) -> [x[1], x[2]]`. This will be stored in `contres.branch` where `contres::AbstractBranchResult` is the continuation curve of the bifurcation problem. Finally, the first component is used for plotting in the continuation curve.
- `save_solution::Any`: Function to save the full solution on the branch. Some problem are updated during computation (like periodic orbit functional with adaptive mesh) and this function allows to save the state of the problem along with the solution itself. Note that this should allocate the output (i.e. not as a view). Signature: `save_solution(x, p)`. Defaults to `save_solution_default(x, p) = x`.
- `update!::Any`: Function to update the problem after each continuation step. Defaults to `update_default`. It has signature `update!(prob, iter, state)` where `prob` is the current bifurcation problem, `iter::ContIterable` the current iterable and `state::ContState` the current state of the continuation. It should return `true` if the update was successful and `false` otherwise. The continuation will stop if `false` is returned. This type of function is useful for example to update the problem internals like meshes, preconditioners, etc.

`BifurcationKit.DAEBifProblem` — Type

```
struct DAEBifProblem{Tvf, Tu, Tp, Tl<:Union{typeof(identity), IndexLens, PropertyLens, ComposedFuncti
```

Structure to hold a bifurcation problem. Specific to Differential Algebraic Equations (DAE). The options are set accordingly. 🚧🚧 This is work in progress 🚧🚧.

Methods

- `re_make(pb; kwargs...)` modify a bifurcation problem
- `getu0(pb)` calls `pb.u0`
- `getparams(pb)` calls `pb.params`
- `getlens(pb)` calls `pb.lens`
- `getparam(pb)` calls `get(pb.params, pb.lens)`
- `setparam(pb, p0)` calls `set(pb.params, pb.lens, p0)`
- `record_from_solution(pb)` calls `pb.recordFromSolution`
- `plot_solution(pb)` calls `pb.plotSolution`
- `is_symmetric(pb)` calls `is_symmetric(pb.prob)`
- `getdelta(prob)`

Constructors

- `DAEBifProblem(F, u0, params, lens)` all derivatives are computed using ForwardDiff.
- `DAEBifProblem(F, u0, params, lens; J, Jᵗ, d2F, d3F, kwargs...)` and `kwargs` are the fields above. You can pass your own jacobian with `J` (see `BifFunction` for description of the jacobian function) and jacobian adjoint with

- `record_from_solution` see above
- `plot_solution` see above
- `issymmetric[=false]` whether the jacobian is symmetric, this remove the need of providing an adjoint
- `jvp` jacobian-vector product, signature `jvp(x, p, dx)`
- `vjp` vector-jacobian product (adjoint of jvp), signature `vjp(x, p,dx)`
- `d2F` second Differential of `F` with respect to `x`, signature `d2F(x, p, dx1, dx2)`
- `d3F` third Differential of `F` with respect to `x`, signature `d3F(x, p, dx1, dx2, dx3)`
- `save_solution` specify a particular way to record solution which are written in `br.sol`. This can be useful in very particular situations and we recommend using `record_from_solution` instead. For example, it is used internally to record the mesh in the collocation method because this mesh can be modified.

Internal fields

- `VF::Any`: Vector field, typically a `BifFunction`.
- `u0::Any`: Initial guess.
- `params::Any`: Parameters.
- `lens::Union{typeof(identity), IndexLens, PropertyLens, ComposedFunction}`: Typically a `Accessors.PropertyLens`. It specifies which parameter axis among `params` is used for continuation. For example, if `par = (α = 1.0, β = 1.78)`, we can perform continuation w.r.t. `α` by using `lens = (@optic _.α)`. If you have an array `par = [1.0, 2.0]` and want to perform continuation w.r.t. the first variable, you can use `lens = (@optic _[1])` or pass directly `lens = 1`. For more information, we refer to `Accessors.jl`.
- `plotSolution::Any`: user function to plot solutions during continuation. Signature: `plot_solution(x, p; kwargs...)` for `Plot.jl` and `plot_solution(ax, x, p; ax1 = nothing, kwargs...)` for the `Makie.jl`.
- `recordFromSolution::Any`: `record_from_solution = (x, p; k...) -> norm(x)` function used to record a few indicators about the solution. It could be `norm` or `(x, p; k...) -> x[1]`. This is also useful when saving several huge vectors is not possible for memory reasons (for example on GPU). This function can return pretty much everything but you should keep it small. For example, you can do `(x, p; k...) -> (x1 = x[1], x2 = x[2], nrm = norm(x))` or simply `(x, p; k...) -> (sum(x), 1)` or `(x, p; k...) -> [x[1], x[2]]`. This will be stored in `contres.branch` where `contres::AbstractBranchResult` is the continuation curve of the bifurcation problem. Finally, the first component is used for plotting in the continuation curve.
- `save_solution::Any`: Function to save the full solution on the branch. Some problem are updated during computation (like periodic orbit functional with adaptive mesh) and this function allows to save the state of the problem along with the solution itself. Note that this should allocate the output (i.e. not as a view). Signature: `save_solution(x, p)`. Defaults to `save_solution_default(x, p) = x`.
- `update!::Any`: Function to update the problem after each continuation step. Defaults to `update_default`. It has signature `update!(prob, iter, state)` where `prob` is the current bifurcation problem, `iter::ContIterable` the current iterable and `state::ContState` the current state of the continuation. It should return `true` if the update was successful and `false` otherwise. The continuation will stop if `false` is returned. This type of function is useful for example to update the problem internals like meshes, preconditioners, etc.

Missing docstring.

Missing docstring for `PDEBifProblem`. Check Documenter's build log for details.

`BifurcationKit.DeflationOperator` — Type

```
struct DeflationOperator{Tp<:Real, Tdot, T<:Real, vectype} <: BifurcationKit.AbstractDeflationFactor
```

This operator allows to handle the following situation. Assume you want to solve `F(x)=0` with a Newton algorithm but you want to avoid the process to return some already known solutions $roots_i$. The deflation operator penalizes these roots. You can create a `DeflationOperator` to define a scalar function `M(u)` used to find, with Newton iterations, the zeros of the following function

where $||u||_2 = dot(u,u)$. The fields of the struct `DeflationOperator` are as follows:

GitHub

Internal fields

- `power::Real`: power `p`. You can use an `Int` for example.
- `dot::Any`: function, this function has to be bilinear and symmetric for the linear solver to work well.
- `α::Real`: shift.
- `roots::Vector`: roots.
- `tmp::Any`: [internal] to reduce allocations during computation.
- `autodiff::Bool`: [internal] to reduce allocations during computation.
- `δ::Real`: [internal] for finite differences.

Given `defOp::DeflationOperator`, one can access its roots via `defOp[n]` as a shortcut for `defOp.roots[n]`. Note that you can also use `defOp[end]`.

Also, one can add (resp. remove) a new root by using `push!(defOp, newroot)` (resp. `pop!(defOp)`). Finally `length(defOp)` is a shortcut for `length(defOp.roots)`

Constructors

- `DeflationOperator(p::Real, α::Real, roots::Vector{vectype}; autodiff = false)`
- `DeflationOperator(p::Real, dt, α::Real, roots::Vector{vectype}; autodiff = false)`
- `DeflationOperator(p::Real, α::Real, roots::Vector{vectype}, v::vectype; autodiff = false)`

The option `autodiff` triggers the use of automatic differentiation for the computation of the gradient of the scalar function `M`. This works only on `AbstractVector` for now.

Custom distance

You are asked to pass a scalar product like `dot` to build a `DeflationOperator`. However, in some cases, you may want to pass a custom distance `dist(u, v)`. You can do this using

```
`DeflationOperator(p, CustomDist(dist), α, roots)`
```

Note that passing `CustomDist(dist, true)` will trigger the use of automatic differentiation for the gradient of `M`.

Linear solvers / jacobians

When used with newton, you have access to the following linear solvers

- custom solver `DeflatedProblemCustomLS()` which requires solving two linear systems `J•x = rhs`.
- For other linear solvers `<: AbstractLinearSolver`, a matrix free method is used for the deflated functional.
- if passed `Val(:autodiff)`, then `ForwardDiff.jl` is used to compute the jacobian Matrix of the deflated problem.
- if passed `Val(:fullIterative)`, then a full matrix free method is used for the deflated problem.

`BifurcationKit.DeflatedProblem` — Type

```
pb = DeflatedProblem(prob, M::DeflationOperator, jactype)
```

Create a `DeflatedProblem`.

This creates a deflated functional (problem) $M(u) \cdot F(u) = 0$ where `M` is a `DeflationOperator` which encodes the penalization term. `prob` is an `AbstractBifurcationProblem` which encodes the functional. It is not meant not be used directly albeit by advanced users.

Arguments

- `jactype` selects the jacobian for the newton solve. Can be `Val(:autodiff)`, `Val(:fullIterative)`, `Val(:Custom)`

BifurcationKit.Trapeze — Type

GitHub

```
struct Trapeze{Tprob, vectype, Tls<:BifurcationKit.AbstractLinearSolver, T, Tmass, Tjac} <: Bifurcati
```

This composite type implements Finite Differences based on a Trapeze rule (aka Crank-Nicolson, order 2 in time) to locate periodic orbits / BVP. More details (maths, notations, linear systems) can be found here.

The scheme is as follows. We first consider a partition of $[0, 1]$ given by $0 < s_0 < \cdots < s_m = 1$ and one looks for `T = x[end]` such that

$$M_a \cdot (x_i - x_{i-1}) - \frac{T \cdot h_i}{2} (F(x_i) + F(x_{i-1})) = 0,\ i = 1, \cdots, m-1$$

with $u_0 := u_{m-1}$ and the periodicity condition $u_m - u_1 = 0$ and

where $h_1 = s_i - s_{i-1}$. M_a is a mass matrix. Finally, the phase of the periodic orbit is constrained by using a section (but you could use your own)

$$\sum_i \langle x_i - x_{\pi,i}, \phi_i \rangle = 0.$$

Internal fields

- `prob_vf::Any`: a bifurcation problem Default: nothing
- `ϕ::Any`: used to set a section for the phase constraint equation, of size N*M Default: nothing
- `xπ::Any`: used in the section for the phase constraint equation, of size N*M Default: nothing
- `M::Int64`: number of time slices Default: 0
- `mesh::BifurcationKit.TimeMesh`: Mesh, see `TimeMesh` Default: TimeMesh(M)
- `N::Int64`: dimension of the problem in case of an `AbstractVector` Default: 0
- `linsolver::BifurcationKit.AbstractLinearSolver`: linear solver for each time slice, i.e. to solve `J•sol = rhs`. This is only needed for the computation of the Floquet multipliers in a matrix-free setting. Default: DefaultLS()
- `ongpu::Bool`: whether the computation takes place on the gpu (Experimental) Default: false
- `isautonomous::Bool`: Default: true
- `massmatrix::Any`: a mass matrix. You can pass for example a sparse matrix. Default: identity matrix. Default: nothing
- `update_section_every_step::UInt64`: updates the section every `update_section_every_step` step during continuation Default: 1
- `jacobian::Any`: symbol which describes the type of jacobian used in Newton iterations (see below). Default: Dense()

Constructors

The structure can be created by calling `Trapeze(;kwargs...)`. For example, you can declare such a problem without vector field by doing

```
Trapeze(M = 100)
```

Orbit guess

You will see below that you can evaluate the residual of the functional (and other things) by calling `pb(orbitguess, p)` on an orbit guess `orbitguess`. Note that `orbitguess` must be a vector of size M * N + 1 where N is the number of unknowns in the state space and `orbitguess[M*N+1]` is an estimate of the period T of the limit cycle. More precisely, using the above notations, `orbitguess` must be $orbitguess = [x_1, x_2, \cdots, x_M, T]$.

Note that you can generate this guess from a function solution using `generate_solution` or `generate_ci_problem`.

Functional

A functional, hereby called `G`, encodes this problem. The following methods are available

- `pb(orbitguess, p)` evaluates the functional G on `orbitguess`

`orbitguess` without the constraints. It is called `A_γ` in the docs.

- `pb(Val(:JacFullSparseInplace), J, orbitguess, p)`. Same as `pb(Val(:JacFullSparse), orbitguess, p)` but overwrites `J` inplace. Note that the sparsity pattern must be the same independently of the values of the parameters or of `orbitguess`. In this case, this is significantly faster than `pb(Val(:JacFullSparse), orbitguess, p)`.
- `pb(Val(:JacCyclicSparse), orbitguess, p)` return the sparse cyclic matrix Jc (see the docs) of the jacobian `dG(orbitguess)` at `orbitguess`
- `pb(Val(:BlockDiagSparse), orbitguess, p)` return the diagonal of the sparse matrix of the jacobian `dG(orbitguess)` at `orbitguess`. This allows to design Jacobi preconditioner. Use `blockdiag`.

Jacobian

Specify the choice of the jacobian (and linear algorithm), `jacobian` must belong to `(BifurcationKit.Dense(), BifurcationKit.AutoDiffDense(), BifurcationKit.FullLU(), BifurcationKit.FullMatrixFree(), BifurcationKit.BorderedLU(), BifurcationKit.BorderedMatrixFree(), BifurcationKit.FullSparseInplace(), BifurcationKit.BorderedSparseInplace(), BifurcationKit.AutoDiffMF())`. This is used to select a way of inverting the jacobian `dG` of the functional G.

- For `jacobian = FullLU()`, we use the default linear solver based on a sparse matrix representation of `dG`. This matrix is assembled at each newton iteration. This is the default algorithm. Can be used when the sparsity can change.
- For `jacobian = FullSparseInplace()`, this is the same as for `FullLU()` but the sparse matrix `dG` is updated inplace. This method allocates much less. In some cases, this is significantly faster than using `FullLU()`. Note that this method can only be used if the sparsity pattern of the jacobian is always the same.
- For `jacobian = Dense()`, same as above but the matrix `dG` is dense. It is also updated inplace. This option is useful to study ODE of small dimension.
- For `jacobian = AutoDiffDense()`, evaluate the jacobian using ForwardDiff
- For `jacobian = BorderedLU()`, we take advantage of the bordered shape of the linear solver and use a LU decomposition to invert `dG` using a bordered linear solver.
- For `jacobian = BorderedSparseInplace()`, this is the same as for `BorderedLU()` but the cyclic matrix `dG` is updated inplace. This method allocates much less. In some cases, this is significantly faster than using `:BorderedLU`. Note that this method can only be used if the sparsity pattern of the jacobian is always the same.
- For `jacobian = FullMatrixFree()`, a matrix free linear solver is used for `dG`: note that a preconditioner is very likely required here because of the cyclic shape of `dG` which affects negatively the convergence properties of GMRES.
- For `jacobian = BorderedMatrixFree()`, a matrix free linear solver is used but for `Jc` only (see docs): it means that `options.linsolver` is used to invert `Jc`. These two Matrix-Free options thus expose different part of the jacobian `dG` in order to use specific preconditioners. For example, an ILU preconditioner on `Jc` could remove the constraints in `dG` and lead to poor convergence. Of course, for these last two methods, a preconditioner is likely to be required.
- For `jacobian = AutoDiffMF()`, the evaluation map of the differential is derived using automatic differentiation. Thus, unlike the previous two cases, the user does not need to pass a Matrix-Free differential.

> **GPU call**
>
> For these methods to work on the GPU, for example with `CuArrays` in mode `allowscalar(false)`, we face the issue that the function `_extract_period_fdtrap` won't be well defined because it is a scalar operation. Note that you must pass the option `ongpu = true` for the functional to be evaluated efficiently on the gpu.

`BifurcationKit.Collocation` — Type

```
struct Collocation{Tprob<:Union{Nothing, BifurcationKit.AbstractBifurcationProblem}, Tjac<:Bifurcatio
```

This composite type implements an orthogonal collocation (at Gauss points) method of piecewise polynomials to locate periodic orbits / BVP. More details (maths, notations, linear systems) can be found online.

Internal fields

- `xπ::Any`: Used in the section for the phase constraint equation. Default: nothing
- `∂ϕ::Any`: We store the derivative of ϕ, no need to recompute it each time. Default: nothing
- `N::Int64`: Dimension of the state space. Default: 0
- `isautonomous::Bool`: Default: true
- `massmatrix::Any`: Default: nothing
- `update_section_every_step::UInt64`: Update the section every `update_section_every_step` step during continuation. Default: 1
- `jacobian::BifurcationKit.AbstractJacobianType`: Describes the type of jacobian used in Newton/PALC/etc iterations. See below for more information. Default: DenseAnalytical()
- `mesh_cache::BifurcationKit.MeshCollocationCache`: Cache for collocation. See docs of `MeshCollocationCache`. Default: nothing
- `cache::BifurcationKit.POCollCache`: Default: nothing
- `meshadapt::Bool`: Whether to use mesh adaptation. Default: false
- `verbose_mesh_adapt::Bool`: Verbose mesh adaptation information. Default: false
- `K::Float64`: Parameter for mesh adaptation, control new mesh step size. More precisely, we set $\max(h_i) / \min(h_i) \leq K$ if h_i denotes the time steps. Default: 100

Methods

Here are some useful methods you can apply to `coll::Collocation`:

- `length(coll)` gives the total number of unknowns.
- `size(coll)` returns the triplet `(N, m, Ntst)`.
- `getmesh(coll)` returns the mesh $0 = \tau_1 < \ldots < \tau_{ntst+1} = 1$. This is useful because this mesh is bound to vary during automatic mesh adaptation.
- `get_mesh_coll(coll)` returns the (static) mesh $-1 = \sigma_1 < \ldots < \sigma_{m+1} = 1$.
- `get_times(coll)` returns the vector of times (length `1 + m * Ntst`) at the which the collocation is applied.
- `generate_solution(coll, orbit, period)` generate a guess from a function `t -> orbit(t)` which approximates the periodic orbit.
- `POInterpolation(coll, x)` return a function interpolating the solution `x` using a piecewise polynomials function.
- `getperiod(coll po, p)` return the period of the periodic orbit `po`.

Orbit guess

You can evaluate the residual of the functional (and other things) by calling `coll(orbitguess, p)` on an orbit guess `orbitguess`. Note that `orbitguess` must be of size 1 + N * (1 + m * Ntst) where N is the number of unknowns in the state space and `orbitguess[end]` is an estimate of the period `T` of the limit cycle.

Note that you can generate this guess from a function using `generate_solution` or `generate_ci_problem`.

Jacobian

Specify the choice of the jacobian (and linear algorithm), `jacobian` must belong to `(BifurcationKit.AutoDiffDense(), BifurcationKit.DenseAnalytical(), BifurcationKit.FullSparse(), BifurcationKit.DenseAnalyticalInplace(), BifurcationKit.FullSparseInplace(), BifurcationKit.AutoDiffMF())`.

This is used to select a way of inverting the jacobian dG of the functional G. See website for more information.

Constructors

- `Collocation(Ntst::Int, m::Int; kwargs)` creates an empty functional with `Ntst` and `m`.

Functional

A functional, hereby called `G`, encodes this problem. The following methods are available

- `residual(coll, orbitguess, p)` evaluates the functional G on `orbitguess`

BifurcationKit.Shooting — Type

```
struct Shooting{Tf<:BifurcationKit.AbstractFlow, Tjac<:BifurcationKit.AbstractJacobianType, Ts, Tsect
```

Create a problem to implement the Simple / Parallel Multiple Standard Shooting method to locate periodic orbits / BVP. More details (maths, notations, linear systems) can be found here. The arguments are as described below.

A functional, hereby called `G`, encodes the shooting problem. For example, the following methods are available:

- `residual(pb, orbitguess, par)` evaluates the functional G on `orbitguess`
- `jvp(pb, orbitguess, par, du)` evaluates the jacobian `dG(orbitguess)•du` functional at `orbitguess` on `du`.
- `pb(Val(:JacobianMatrixInplace), J, x, par)` compute the jacobian of the functional analytically. This is based on ForwardDiff.jl. Useful mainly for ODEs.
- `pb(Val(:JacobianMatrix), x, par)` same as above but out-of-place.

You can then call `residual(pb, orbitguess, par)` to apply the functional to a guess. Note that you can generate this guess from a function solution using `generate_solution` or `generate_ci_problem`.

Allowed types

- `orbitguess::AbstractVector` must be of size `M * N + 1` where N is the number of unknowns of the state space and `orbitguess[M * N + 1]` is an estimate of the period `T` of the limit cycle. This form of guess is convenient for the use of the linear solvers in `IterativeSolvers.jl` (for example) which only accept `AbstractVectors`.
- `orbitguess::BorderedArray(guess, T)` where `guess[i]` is the state of the orbit at the `i`th time slice. This last form allows for non-vector state space which can be convenient for 2d problems for example, use `GMRESKrylovKit` for the linear solver in this case.

Internal fields

- `M::Int64`: `ds`: vector of time differences for each shooting. Its length is written `M`. If `M == 1`, then the simple shooting is implemented and the multiple one otherwise. Default: 0
- `flow::BifurcationKit.AbstractFlow`: `flow::Flow`: implements the flow of the Cauchy problem though the structure `Flow`. Default: Flow()
- `ds::Any`: Default: diff(LinRange(0, 1, M + 1))
- `section::Any`: `section`: implements a phase condition. The evaluation `section(x, T)` must return a scalar number where `x` is a guess for **one point** on the periodic orbit and `T` is the period of the guess. Also, the method `section(x, T, dx, dT)` must be available and must return the differential of `section`. The type of `x` depends on what is passed to the newton solver. See `SectionSS` for a type of section defined as a hyperplane. Default: nothing
- `parallel::Bool`: `parallel` whether the shooting is computed in parallel (threading). Available through the use of Flows defined by `EnsembleProblem` (this is automatically set up for you). Default: false
- `par::Any`: `par` parameters of the model Default: nothing
- `lens::Any`: `lens` parameter axis. Default: nothing
- `update_section_every_step::UInt64`: updates the section every `update_section_every_step` step during continuation Default: 1
- `jacobian::BifurcationKit.AbstractJacobianType`: Describes the type of jacobian used in Newton iterations (see below). Default: AutoDiffDense()

Jacobian

`jacobian` Specify the choice of the linear algorithm, which must belong to `(BifurcationKit.AutoDiffMF(), BifurcationKit.MatrixFree(), BifurcationKit.AutoDiffDense(), BifurcationKit.AutoDiffDenseAnalytical(), BifurcationKit.FiniteDifferences(), BifurcationKit.FiniteDifferencesMF())`. This is used to select a way of inverting the jacobian dG

```
3. For `AutodiffDense()`. Same as for `AutoDiffMF` but the jacobian is formed as a dense Matrix. You
4. For `FiniteDifferences()`, same as for `AutoDiffDense` but we use Finite Differences to compute th
5. For `AutoDiffDenseAnalytical()`. Same as for `AutoDiffDense` but the jacobian is formed using a mi
6. For `FiniteDifferencesMF()`, use Finite Differences to compute the matrix-free jacobian of `x -> p
```

Simplified constructors

- The first important constructor is the following which is used for branching to periodic orbits from Hopf bifurcation points:

```
pb = Shooting(M::Int, prob::Union{ODEProblem, EnsembleProblem}, alg; kwargs...)
```

- A convenient way to build the functional is to use:

```
pb = Shooting(prob::Union{ODEProblem, EnsembleProblem}, alg, centers::AbstractVector; kwargs...)
```

where `prob` is an `ODEProblem` (resp. `EnsembleProblem`) which is used to create a flow using the ODE solver `alg` (for example `Tsit5()`). `centers` is list of `M` points close to the periodic orbit, they will be used to build a constraint for the phase. `parallel = false` is an option to use Parallel simulations (Threading) to simulate the multiple trajectories in the case of multiple shooting. This is efficient when the trajectories are relatively long to compute. Finally, the arguments `kwargs` are passed to the ODE solver defining the flow. Look at `DifferentialEquations.jl` for more information. Note that, in this case, the derivative of the flow is computed internally using Finite Differences.

- Another way to create a Shooting problem with more options is the following where in particular, one can provide its own scalar constraint `section(x)::Number` for the phase:

```
pb = Shooting(prob::Union{ODEProblem, EnsembleProblem}, alg, M::Int, section; parallel = false, kwarg
```

or

```
pb = Shooting(prob::Union{ODEProblem, EnsembleProblem}, alg, ds, section; parallel = false, kwargs...
```

- The next way is an elaboration of the previous one

```
pb = Shooting(prob1::Union{ODEProblem, EnsembleProblem}, alg1, prob2::Union{ODEProblem, EnsembleProbl
```

or

```
pb = Shooting(prob1::Union{ODEProblem, EnsembleProblem}, alg1, prob2::Union{ODEProblem, EnsembleProbl
```

where we supply now two `ODEProblems`. The first one `prob1`, is used to define the flow associated to `F` while the second one is a problem associated to the derivative of the flow. Hence, `prob2` must implement the following vector field $\tilde{F}(x, y, p) = (F(x, p), dF(x, p) \cdot y)$.

`BifurcationKit.PoincareShooting` — Type

```
struct PoincareShooting{Tf, Tjac<:BifurcationKit.AbstractJacobianType, Tsection<:SectionPS, Tpar, Tle
```

Internal fields

GitHub

- `M::Int64`: `M`: the number of Poincaré sections. If `M == 1`, then the simple shooting is implemented and the multiple one otherwise. Default: 0
- `flow::Any`: `flow::Flow`: implements the flow of the Cauchy problem though the structure `Flow`. Default: Flow()
- `section::SectionPS`: `sections`: function or callable struct which implements a Poincaré section condition. The evaluation `sections(x)` must return a scalar number when `M == 1`. Otherwise, one must implement a function `section(out, x)` which populates `out` with the `M` sections. See `SectionPS` for type of section defined as a hyperplane. Default: SectionPS(M)
- `δ::Float64`: `δ = 1e-8` used to compute the jacobian of the functional by finite differences. If set to `0`, an analytical expression of the jacobian is used instead. Default: 0.0
- `parallel::Bool`: `parallel = false` whether the shooting are computed in parallel (threading). Only available through the use of Flows defined by `EnsembleProblem`. Default: false
- `par::Any`: `par` parameters of the model Default: nothing
- `lens::Any`: `lens` parameter axis Default: nothing
- `update_section_every_step::UInt64`: `update_section_every_step` updates the section every `update_section_every_step` step during continuation Default: 1
- `jacobian::BifurcationKit.AbstractJacobianType`: Describes the type of jacobian used in Newton iterations (see below). Default: AutoDiffDenseAnalytical()

Jacobian

`jacobian` Specify the choice of the linear algorithm, which must belong to `(BifurcationKit.AutoDiffMF(), BifurcationKit.MatrixFree(), BifurcationKit.AutoDiffDense(), BifurcationKit.AutoDiffDenseAnalytical(), BifurcationKit.FiniteDifferences(), BifurcationKit.FiniteDifferencesMF())`. This is used to select a way of inverting the jacobian dG

```
1. For `MatrixFree()`, matrix free jacobian, the jacobian is specified by the user in `prob`. This is
2. For `AutoDiffMF()`, we use Automatic Differentiation (AD) to compute the (matrix-free) derivative
3. For `AutodiffDense()`. Same as for `AutoDiffMF` but the jacobian is formed as a dense Matrix. You
4. For `FiniteDifferences()`, same as for `AutoDiffDense` but we use Finite Differences to compute th
5. For `AutoDiffDenseAnalytical()`. Same as for `AutoDiffDense` but the jacobian is formed using a mi
6. For `FiniteDifferencesMF()`, use Finite Differences to compute the matrix-free jacobian of `x -> p
```

Simplified constructors

- The first important constructor is the following which is used for branching to periodic orbits from Hopf bifurcation points pb = PoincareShooting(M::Int, prob::Union{ODEProblem, EnsembleProblem}, alg; kwargs...)
- A convenient way is to create a functional is

```
pb = PoincareShooting(prob::ODEProblem, alg, section; kwargs...)
```

for simple shooting or

```
pb = PoincareShooting(prob::Union{ODEProblem, EnsembleProblem}, alg, M::Int, section; kwargs...)
```

for multiple shooting . Here `prob` is an `Union{ODEProblem, EnsembleProblem}` which is used to create a flow using the ODE solver `alg` (for example `Tsit5()`). Finally, the arguments `kwargs` are passed to the ODE solver defining the flow. We refer to `DifferentialEquations.jl` for more information.

- Another convenient call is

```
pb = PoincareShooting(prob::Union{ODEProblem, EnsembleProblem}, alg, normals::AbstractVector,
centers::AbstractVector; δ = 1e-8, kwargs...)
```

where `normals` (resp. `centers`) is a list of normals (resp. centers) which defines a list of hyperplanes Σ_i. These hyperplanes are used to define partial Poincaré return maps.

A functional, hereby called G encodes this shooting problem. You can then call `residual(pb, orbitguess, par)` to apply the functional to a guess. Note that `orbitguess::AbstractVector` must be of size M * N where N is the number of unknowns in the state space and `M` is the number of Poincaré maps. Another accepted `guess` is such that `guess[i]` is the state of the orbit on the `i`th section. This last form allows for non-vector state space which can be convenient for 2d problems for example.

Note that you can generate this guess from a function solution using `generate_solution`.

- `residual(pb, orbitguess, par)` evaluates the functional G on `orbitguess`
- `jvp(pb, orbitguess, par, du)` evaluates the jacobian `dG(orbitguess).du` functional at `orbitguess` on `du`
- `pb`(Val(:JacobianMatrixInplace), J, x, par)` compute the jacobian of the functional analytically. This is based on ForwardDiff.jl. Useful mainly for ODEs.
- `pb(Val(:JacobianMatrix), x, par)` same as above but out-of-place.

Tip

You can use the function `getperiod(pb, sol, par)` to get the period of the solution `sol` for the problem with parameters `par`.

source

Waves

`BifurcationKit.TWModel` — Type

Missing docstring.

Missing docstring for `BifurcationKit.EigenWave`. Check Documenter's build log for details.

Missing docstring.

Missing docstring for `BifurcationKit.GEigenWave`. Check Documenter's build log for details.

Missing docstring.

Missing docstring for `continuation(prob::BifurcationKit.TWModel, orbitguess, alg::BifurcationKit.AbstractContinuationAlgorithm, contParams::BifurcationKit.ContinuationPar; kw...)`. Check Documenter's build log for details.

Linear solvers

`BifurcationKit.DefaultLS` — Type

```
struct DefaultLS <: BifurcationKit.AbstractDirectLinearSolver
```

This struct is used to provide the backslash operator `. Can be used to solve` $(a_0 * I + a_1 * J) * x = rhs$`.

Internal fields

- `useFactorization::Bool`: Whether to catch a factorization for multiple solves. Some operators may not support LU (like ApproxFun.jl) or QR factorization so it is best to let the user decides. Some matrices do not have `factorize` like `StaticArrays.MMatrix`. Default: true

`BifurcationKit.DefaultPILS` — Type

[Mainly for debugging] This solver is used to test Moore-Penrose continuation. This is defined as an iterative pseudo-inverse linear solver. Used to solve `J * x = rhs`.

Internal fields

- `useFactorization::Bool`: Whether to catch a factorization for multiple solves. Some operators may not support LU (like ApproxFun.jl) or QR factorization so it is best to let the user decides. Some matrices do not have `factorize` like `StaticArrays.MMatrix`. Default: true

BifurcationKit.GMRESIterativeSolvers — Type

```
mutable struct GMRESIterativeSolvers{T, Tl, Tr} <: BifurcationKit.AbstractIterativeLinearSolver
```

Linear solver based on `gmres` from `IterativeSolvers.jl`. Can be used to solve `(a₀ * I + a₁ * J) * x = rhs`.

The struct is mutable so that you can modify the preconditioners.

Internal fields

- `abstol::Any`: Absolute tolerance for solver Default: 0.0
- `reltol::Any`: Relative tolerance for solver Default: 1.0e-8
- `restart::Int64`: Number of restarts Default: 200
- `maxiter::Int64`: Maximum number of iterations Default: 100
- `N::Int64`: Dimension of the problem Default: 0
- `verbose::Bool`: Display information during iterations Default: false
- `log::Bool`: Record information Default: true
- `initially_zero::Bool`: Start with zero guess Default: true
- `Pl::Any`: Left preconditioner Default: IterativeSolvers.Identity()
- `Pr::Any`: Right preconditioner Default: IterativeSolvers.Identity()
- `ismutating::Bool`: Whether the linear operator is written inplace Default: false

BifurcationKit.GMRESKrylovKit — Type

```
mutable struct GMRESKrylovKit{𝒯, 𝒯l} <: BifurcationKit.AbstractIterativeLinearSolver
```

Create a linear solver based on `linsolve` from `KrylovKit.jl`. Can be used to solve `(a₀ * I + a₁ * J) * x = rhs`.

The struct is mutable so that you can modify the preconditioners.

Different linear solvers

By tuning the options, you can select CG, GMRES... see here

Internal fields

- `dim::Int64`: Krylov Dimension Default: KrylovDefaults.krylovdim[]
- `atol::Any`: Absolute tolerance for solver Default: KrylovDefaults.tol[]
- `rtol::Any`: Relative tolerance for solver Default: KrylovDefaults.tol[]
- `maxiter::Int64`: Maximum number of iterations Default: KrylovDefaults.maxiter[]
- `verbose::Int64`: Verbosity ∈ {0,1,2} Default: 0
- `issymmetric::Bool`: If the linear map is symmetric, only meaningful if T<:Real Default: false
- `ishermitian::Bool`: If the linear map is hermitian Default: false
- `isposdef::Bool`: If the linear map is positive definite Default: false

BifurcationKit.KrylovLS — Type

GitHub

```
mutable struct KrylovLS{K, 𝒯l, 𝒯r} <: BifurcationKit.AbstractIterativeLinearSolver
```

Create a linear solver based on Krylov.jl. Can be used to solve `(a₀ * I + a₁ * J) * x = rhs`. You have access to `cg, cr, gmres, symmlq, cg_lanczos, cg_lanczos_shift_seq`...

The struct is mutable so that you can modify the preconditioners.

Example

You can create a Krylov solver with the following code:

```
KrylovLS(atol = 1e-11, rtol = 1e-8)
```

Other methods

Look at `KrylovLSInplace` for a method where the Krylov space is kept in memory

Internal fields

- `KrylovAlg::Symbol`: Krylov method
- `kwargs::Any`: Arguments passed to the linear solver
- `Pl::Any`: Left preconditioner
- `Pr::Any`: Right preconditioner

BifurcationKit.KrylovLSInplace — Type

```
mutable struct KrylovLSInplace{F, K, 𝒯l, 𝒯r} <: BifurcationKit.AbstractIterativeLinearSolver
```

Create an inplace linear solver based on Krylov.jl. Can be used to solve `(a₀ * I + a₁ * J) * x = rhs`.

The Krylov space is pre-allocated. This is really great for GPU but also for CPU.

The struct is mutable so that you can modify the preconditioners.

Internal fields

- `workspace::Any`: Can be Krylov.GmresWorkspace for example.
- `KrylovAlg::Symbol`: Krylov method.
- `kwargs::Any`: Arguments passed to the linear solver.
- `Pl::Any`: Left preconditioner.
- `Pr::Any`: Right preconditioner.
- `is_inplace::Bool`: Is the linear mapping inplace.

Eigen solvers

BifurcationKit.DefaultEig — Type

```
struct DefaultEig{T} <: BifurcationKit.AbstractDirectEigenSolver
```

The struct `DefaultEig` is used to provide the `eigen` method to `BifurcationKit`.

Internal fields

- `which::Any`: How do we sort the computed eigenvalues. Default: real

Just pass the above fields like `DefaultEig(; which = abs)`

GitHub

BifurcationKit.EigArpack — Type

```
struct EigArpack{T, Tby, Tw} <: BifurcationKit.AbstractIterativeEigenSolver
```

Create an eigen solver based on Arpack.jl.

Internal fields

- `sigma::Any`: Shift for Shift-Invert method with `(J - sigma ⋅ I)
- `which::Symbol`: Which eigen-element to extract :LR, :LM, ...
- `by::Any`: Sorting function, default to real
- `kwargs::Any`: Keyword arguments passed to EigArpack

Constructor

```
EigArpack(sigma = nothing, which = :LR; kwargs...)
```

More information is available at Arpack.jl. You can pass the following parameters `tol = 0.0, maxiter = 300, ritzvec = true, v0 = zeros((0,))`.

BifurcationKit.EigKrylovKit — Type

```
struct EigKrylovKit{T, vectype} <: BifurcationKit.AbstractMFEigenSolver
```

Create an eigen solver based on `KrylovKit.jl`.

Internal fields

- `dim::Int64`: Krylov Dimension Default: KrylovDefaults.krylovdim[]
- `tol::Any`: Tolerance Default: 0.0001
- `restart::Int64`: Number of restarts Default: 200
- `maxiter::Int64`: Maximum number of iterations Default: KrylovDefaults.maxiter[]
- `verbose::Int64`: Verbosity ∈ {0, 1, 2} Default: 0
- `which::Symbol`: Which eigenvalues are looked for :LR (largest real), :LM, ... Default: :LR
- `issymmetric::Bool`: If the linear map is symmetric, only meaningful if T<:Real Default: false
- `ishermitian::Bool`: If the linear map is hermitian Default: false
- `x₀::Any`: Example of vector to usen for Krylov iterations Default: nothing

Constructors

Just pass the above fields like `EigKrylovKit(;dim=2)`

BifurcationKit.EigArnoldiMethod — Type

```
struct EigArnoldiMethod{T, Tby, Tw, Tkw, vectype} <: BifurcationKit.AbstractIterativeEigenSolver
```

Internal fields

- `sigma::Any`: Shift for Shift-Invert method.
- `which::Any`: Which eigen-element to extract LR(), LM(), ...
- `by::Any`: How do we sort the computed eigenvalues, defaults to real.
- `kwargs::Any`: Key words arguments passed to EigArpack.
- `x₀::Any`: Example of vector used for Krylov iterations.

Constructor

```
EigArnoldiMethod(;sigma = nothing, which = ArnoldiMethod.LR(), x₀ = nothing, kwargs...)
```

BifurcationKit.EigenMassMatrix — Type

```
struct EigenMassMatrix{Tb, Teig<:AbstractEigenSolver} <: AbstractEigenSolver
```

Create an eigensolver for DAE, Basically a GEV with mass matrix.

Internal fields

- `B::Any`: Mass matrix
- `eig::AbstractEigenSolver`: Eigen-solver

Generalized Eigen solvers

Missing docstring.

Missing docstring for `BifurcationKit.gev`. Check Documenter's build log for details.

BifurcationKit.DefaultGEig — Type

The struct `Default` is used to provide the backslash operator to our Package

BifurcationKit.GEigArpack — Type

```
struct GEigArpack{T, Tby, Tw, Tb} <: BifurcationKit.AbstractGEigenSolver
```

Internal fields

- `sigma::Any`: Shift for Shift-Invert method with `(J - sigma⋅I)
- `which::Symbol`: Which eigen-element to extract :LR, :LM, ...
- `by::Any`: Sorting function, default to real
- `kwargs::Any`: Keyword arguments passed to EigArpack
- `B::Any`: Mass matrix

More information is available at Arpack.jl. You can pass the following parameters `tol=0.0, maxiter=300, ritzvec=true, v0=zeros((0,))`.

Constructor

```
EigArpack(sigma = nothing, which = :LR; kwargs...)
```

Missing docstring.

Missing docstring for `BifurcationKit.GEigKrylovKit`. Check Documenter's build log for details.

Missing docstring.

Missing docstring for `BifurcationKit.GEigArnoldiMethod`. Check Documenter's build log for details.

Floquet solvers

```
struct FloquetQaD{E<:AbstractEigenSolver} <: BifurcationKit.AbstractFloquetSolver
```
GitHub

Computes Floquet multipliers (eigenvalues of the monodromy matrix) for periodic orbit problems using the Shooting method or Finite Differences (Trapeze method).

Method Description

The "Quick and Dirty" (QaD) method computes Floquet multipliers through sequential matrix products along the periodic orbit. This approach has numerical limitation:

- **Precision issues**: Accuracy degrades for large or small Floquet exponents when using many time sections, due to accumulated errors from repeated matrix multiplications.

Despite precision limitations, the method is sufficient for bifurcation detection in most cases.

Internal fields

- `eigsolver::AbstractEigenSolver`: eigensolver used to compute the eigenvalues of the monodromy matrix.
- `matrix_free::Bool`: whether to use a matrix-free linear operator (automatic when `eigsolver` is not a direct solver).

Implementation Details

- If `eigsolver == DefaultEig()`, the full monodromy matrix is explicitly formed and its eigenvalues are computed
- Otherwise, a matrix-free formulation is used, applying the monodromy operator via sequential time evolution

Floquet multipliers computation

The computation of Floquet multipliers is necessary for the detection of bifurcations of periodic orbits (which is done by analyzing the Floquet exponents obtained from the Floquet multipliers). Hence, the eigensolver `eigsolver` needs to compute the eigenvalues with largest modulus (and not the ones with largest real part which is their default behavior). This can be done by changing the option `which = :LM` of `eigsolver`. Nevertheless, note that for most implemented eigensolvers in `BifurcationKit`, the proper option is properly set.

Collocation

`BifurcationKit.FloquetGEV` — Type

```
struct FloquetGEV{E<:AbstractEigenSolver, Tb} <: BifurcationKit.AbstractFloquetSolver
```

Computes Floquet exponents for periodic orbits using a Generalized Eigenvalue Problem (GEV) formulation.

Method Description

This method reformulates the Floquet multiplier computation as a large-dimensional generalized eigenvalue problem. The approach is based on a simplified version of the algorithm described in [1].

Performance characteristics:

- **Accuracy**: More numerically precise than `FloquetQaD`, especially for extreme Floquet exponents.
- **Speed**: Slower due to the large dimension of the eigenvalue problem.
- **Recommendation**: Use `FloquetColl()` when possible for better performance with similar accuracy.

Fields

- `eigsolver::AbstractEigenSolver`: Eigensolver used to solve the generalized eigenvalue problem.
- `B::Tb`: Mass matrix for the generalized eigenvalue formulation.

Constructor

Arguments:
GitHub

- `eigls`: Eigensolver to use.
- `ntot`: Total dimension of the generalized eigenvalue problem.
- `n`: State space dimension.
- `array_zeros`: Function to allocate zero arrays: defaults to `zeros` but `spzeros` can be passed for sparse matrices.

Example

```
# For a 2D system with 30 time sections and 4 collocation points:
eigfloquet = FloquetGEV(DefaultEig(), (30 * 4 + 1) * 2, 2)
```

References

[1] Fairgrieve, Thomas F., and Allan D. Jepson. “O. K. Floquet Multipliers.” SIAM Journal on Numerical Analysis 28, no. 5 (October 1991): 1446–62. https://doi.org/10.1137/0728075.

BifurcationKit.FloquetColl — Type

```
struct FloquetColl{E<:AbstractEigenSolver, C} <: BifurcationKit.AbstractFloquetSolver
```

Computes Floquet exponents for periodic orbits discretized with the orthogonal collocation method.

Method Description

This method uses the **condensation of parameters** technique, originally described in [1] and implemented in AUTO07p, with improvements from [2,3]. The approach efficiently computes Floquet multipliers by exploiting the structure of the collocation discretization.

Performance characteristics:

- **Accuracy**: High precision, comparable to `FloquetGEV`.
- **Speed**: Faster than `FloquetGEV` by exploiting collocation structure.
- **Recommendation**: Preferred method for collocation-based periodic orbit problems.

Fields

- `eigsolver::AbstractEigenSolver`: Which eigen solver. Defaults to `DefaultEig`.
- `cache::Any`: Cache, defaults to `nothing`. It should be set to `COPCACHE`. When used with `COPBLS`, it is automatically set up.
- `small_n::Bool`: Whether to use optimized COP to compute the Floquet exponents. Note that this does not work well for sparse matrices. Defaults to `true`.

Constructor

```
FloquetColl(; eigls::AbstractEigenSolver = DefaultEig(), cache = nothing, small_n = true)
```

Keyword Arguments:

- `eigls`: Eigensolver to use (defaults to `DefaultEig()`)
- `cache`: Optional cache for optimization (set to `COPCACHE`; automatically configured when using `COPBLS`)
- `small_n`: Whether to use optimized algorithm for small state dimensions (defaults to `true`)

References

[1] Doedel, Eusebius, Herbert B. Keller, et Jean Pierre Kernevez. «Numerical analysis and control of bifurcation problems (ii): bifurcation in infinite dimensions». International Journal of Bifurcation and Chaos 01, n° 04 (décembre 1991): 745-72. https://doi.org/10.1142/S0218127491000555.

[3] Fairgrieve, Thomas F., and Allan D. Jepson. “O. K. Floquet Multipliers.” SIAM Journal on Numerical Analysis 28, no. 5 (October 1991): 1446–62. https://doi.org/10.1137/0728075.

Bordered linear solvers

BifurcationKit.MatrixBLS — Type

```
struct MatrixBLS{S<:Union{Nothing, BifurcationKit.AbstractLinearSolver}} <: BifurcationKit.AbstractBo
```

This struct is used to provide the bordered linear solver based on inverting the full matrix.

Internal fields

- `solver::Union{Nothing, BifurcationKit.AbstractLinearSolver}`: Linear solver used to invert the full matrix.

BifurcationKit.BorderingBLS — Type

```
struct BorderingBLS{S<:Union{Nothing, BifurcationKit.AbstractLinearSolver}, Ttol<:Real, Tdot, Tnorm}
```

This struct is used to provide the bordered linear solver based on the Bordering method. Using the options, you can trigger a sequence of Bordering reductions to meet a given precision.

Internal fields

- `solver::Union{Nothing, BifurcationKit.AbstractLinearSolver}`: Linear solver for the Bordering method. Default: nothing
- `tol::Real`: Tolerance for checking precision. Default: 1.0e-12
- `check_precision::Bool`: Check precision of the linear solve? Default: true
- `k::Int64`: Number of recursions to achieve tolerance. Default: 1
- `dot::Any`: Inner product used by the solver. Default: VI.inner
- `norm::Any`: Norm used by the solver. Default: VI.norm

Constructors

- there is a simple constructor `BorderingBLS(ls)` where `ls` is a linear solver, for example `ls = DefaultLS()`
- you can pass keyword arguments to create the solver, for example `BorderingBLS(solver = DefaultLS(), tol = 1e-4)`

Reference(s)

This is the solver BEC + k in:

Govaerts, W. “Stable Solvers and Block Elimination for Bordered Systems.” SIAM Journal on Matrix Analysis and Applications 12, no. 3 (July 1, 1991): 469–83. https://doi.org/10.1137/0612034.

BifurcationKit.MatrixFreeBLS — Type

```
struct MatrixFreeBLS{S<:Union{Nothing, BifurcationKit.AbstractLinearSolver}} <: BifurcationKit.Abstra
```

This struct is used to provide a bordered linear solver based on a matrix free operator for the full system in `(x, p)`.

```
MatrixFreeBLS(solver, ::Bool)
```

GitHub

Internal fields

- `solver::Union{Nothing, BifurcationKit.AbstractLinearSolver}`: Linear solver for solving the extended linear system.
- `use_bordered_array::Bool`: Structure used to hold `(x, p)`. If `true`, this is achieved using `BorderedArray`. If `false`, a `Vector` is used which is analogous to `vcat(x, p)`.

`BifurcationKit.MatrixFreeBLSmap` — Type

```
struct MatrixFreeBLSmap{Tj, Ta, Tb, Tc, Ts, Td}
```

Composite type to save the bordered linear system with expression

┌ ┐ | J a | | b' c | └ ┘

It then solved using Matrix Free algorithm applied to the full operator and not just J as for MatrixFreeBLS

`BifurcationKit.LSFromBLS` — Type

```
struct LSFromBLS{Ts} <: BifurcationKit.AbstractLinearSolver
```

This structure is used to provide the following linear solver. To solve (1) J ⋅ x = rhs, one decomposes J using Matrix by blocks and then use a bordering strategy to solve (1).

> It is interesting for solving the linear system associated with Collocation / Trapeze functionals, for example using `BorderingBLS(solver = BK.LSFromBLS(), tol = 1e-9, k = 2, check_precision = true)`

Warning

The solver only works for `AbstractMatrix`

Internal fields

- `solver::Any`: Linear solver used to solve the smaller linear systems.

Nonlinear solver

`BifurcationKit.solve` — Function

```
solve(prob::AbstractBifurcationProblem, ::Newton, options::NewtonPar; normN = norm, callback = (;x, f
```

This is the Newton-Krylov Solver for `F(x, p0) = 0` with Jacobian w.r.t. `x` written `J(x, p0)` and initial guess `x0`. It is important to set the linear solver `options.linsolver` properly depending on your problem. This linear solver is used to solve $J(x, p_0)u = -F(x, p_0)$ in the Newton step. You can for example use `linsolver = DefaultLS()` which is the operator backslash: it works well for Sparse / Dense matrices. See Linear solvers (LS) for more information.

Arguments

- `prob` a `::AbstractBifurcationProblem`, typically a `BifurcationProblem` which holds the vector field and its jacobian. We also refer to `BifFunction` for more details.

Optional Arguments

GitHub

- `normN = norm` specifies a norm for the convergence criteria
- `callback` function passed by the user which is called at the end of each iteration. The default one is the following `cb_default((x, fx, J, residual, step, itlinear, options, x0, residuals); k...) = true`. Can be used to update a preconditioner for example. You can use for example `cbMaxNorm` to limit the residuals norms. If yo want to specify your own, the arguments passed to the callback are as follows
  - `x` current solution
  - `fx` current residual
  - `J` current jacobian
  - `residual` current norm of the residual
  - `step` current newton step
  - `itlinear` number of iterations to solve the linear system
  - `options` a copy of the argument `options` passed to `newton`
  - `residuals` the history of residuals
  - `kwargs` kwargs arguments, contain your initial guess `x0`

    They can be used as `callback = (state;k...) -> state.residual<1` for example.
- `kwargs` arguments passed to the callback. Useful when `newton` is called from `continuation`

Output:

- `solution::NonLinearSolution`, we refer to `NonLinearSolution` for more information.

Linear solver

Make sure that the linear solver (Matrix-Free...) corresponds to your jacobian (Matrix-Free vs. Matrix based).

```
solve(
    prob::BifurcationKit.AbstractBifurcationProblem,
    defOp::DeflationOperator{Tp, Tdot, T, vectype},
    options::NewtonPar{T, L, E};
    ...
) -> NonLinearSolution{_A, Tprob, Tres} where {_A, Tprob<:(DeflatedProblem{Tprob, _A, _B, _C, _D, Val
solve(
    prob::BifurcationKit.AbstractBifurcationProblem,
    defOp::DeflationOperator{Tp, Tdot, T, vectype},
    options::NewtonPar{T, L, E},
    _linsolver::BifurcationKit.DeflatedProblemCustomLS;
    kwargs...
) -> NonLinearSolution{_A, Tprob, Tres} where {_A, Tprob<:(DeflatedProblem{Tprob, _A, _B, _C, _D, Val
```

This is the deflated version of the Krylov-Newton Solver for `F(x, p0) = 0`.

We refer to the regular `solve` for more information. It penalises the roots saved in `defOp.roots`. The other arguments are as for `solve`. See `DeflationOperator` for more information on `defOp`.

Arguments

Compared to `solve`, the only different arguments are

- `defOp::DeflationOperator` deflation operator
- `linsolver` linear solver used to invert the Jacobian of the deflated functional.
  - custom solver `DeflatedProblemCustomLS()` which requires solving two linear systems `J•x = rhs`.
  - For other linear solvers `<: AbstractLinearSolver`, a matrix free method is used for the deflated functional.
  - if passed `Val(:autodiff)`, then `ForwardDiff.jl` is used to compute the jacobian Matrix of the deflated problem

BifurcationKit.newton — Function

GitHub

This specific Newton-Krylov method first tries to converge to a solution `sol0` close the guess `x0`. It then attempts to converge from the guess `x1` while avoiding the previous converged solution close to `sol0`. This is very handy for branch switching. The method is based on a deflated Newton-Krylov solver.

Arguments

Compared to `newton`, the only different arguments are

- `defOp::DeflationOperator` deflation operator
- `linsolver` linear solver used to invert the Jacobian of the deflated functional.
  - custom solver `DeflatedProblemCustomLS()` which requires solving two linear systems `J•x = rhs`.
  - For other linear solvers `<: AbstractLinearSolver`, a matrix free method is used for the deflated functional.
  - if passed `Val(:autodiff)`, then `ForwardDiff.jl` is used to compute the jacobian Matrix of the deflated problem
  - if passed `Val(:fullIterative)`, then a full matrix free method is used for the deflated problem.

```
newton(
    br::BifurcationKit.AbstractBranchResult,
    ind_bif::Int64;
    normN,
    options,
    start_with_eigen,
    lens2,
    kwargs...
) -> Any
```

This function turns an initial guess for a Fold / Hopf point into a solution to the Fold / Hopf problem based on a Minimally Augmented formulation.

Arguments

- `br` results returned after a call to continuation
- `ind_bif` bifurcation index in `br`

Optional arguments:

- `options::NewtonPar`, default value `br.contparams.newton_options`
- `normN = norm`
- `options` You can pass newton parameters different from the ones stored in `br` by using this argument `options`.
- `bdlinsolver` bordered linear solver for the constraint equation
- `start_with_eigen = false` whether to start the Minimally Augmented problem with information from eigen elements.
- `kwargs` keywords arguments to be passed to the regular Newton-Krylov solver

ODE problems

For ODE problems, it is more efficient to use the Matrix based Bordered Linear Solver passing the option `bdlinsolver = MatrixBLS()`

start_with_eigen

It is recommended that you use the option `start_with_eigen=true`

```
newton(
    disc::BifurcationKit.AbstractShootingDiscretization,
    orbitguess,
```

```
    kwargs...
) -> NonLinearSolution{_A, Tprob, Tres} where {_A, Tprob<:(BifurcationKit.PeriodicOrbitFunctionalSh{T
```

This is the Newton-Krylov Solver for computing a periodic orbit using the (Standard / Poincaré) Shooting method. Note that the linear solver has to be appropriately set up in `options`.

Arguments

Similar to `newton` except that `prob` is either a `Shooting` or a `PoincareShooting`. These two problems have specific options to be tuned, we refer to their link for more information and to the tutorials.

- `prob` a problem of type `<: AbstractShootingDiscretization` encoding the shooting functional G.
- `orbitguess` a guess for the periodic orbit. See `Shooting` and See `PoincareShooting` for information regarding the shape of `orbitguess`.
- `par` parameters to be passed to the functional
- `options` same as for the regular `newton` method.

Optional argument

`jacobian` Specify the choice of the linear algorithm, which must belong to `(BifurcationKit.AutoDiffMF(), BifurcationKit.MatrixFree(), BifurcationKit.AutoDiffDense(), BifurcationKit.AutoDiffDenseAnalytical(), BifurcationKit.FiniteDifferences(), BifurcationKit.FiniteDifferencesMF())`. This is used to select a way of inverting the jacobian dG

```
1. For `MatrixFree()`, matrix free jacobian, the jacobian is specified by the user in `prob`. This is
2. For `AutoDiffMF()`, we use Automatic Differentiation (AD) to compute the (matrix-free) derivative
3. For `AutodiffDense()`. Same as for `AutoDiffMF` but the jacobian is formed as a dense Matrix. You
4. For `FiniteDifferences()`, same as for `AutoDiffDense` but we use Finite Differences to compute th
5. For `AutoDiffDenseAnalytical()`. Same as for `AutoDiffDense` but the jacobian is formed using a mi
6. For `FiniteDifferencesMF()`, use Finite Differences to compute the matrix-free jacobian of `x -> p
```

```
newton(
    disc::BifurcationKit.AbstractShootingDiscretization,
    orbitguess,
    defOp::DeflationOperator{Tp, Tdot, T, vectype},
    options::NewtonPar{T, S, E};
    lens,
    kwargs...
) -> NonLinearSolution{_A, Tprob, Tres} where {_A, Tprob<:(DeflatedProblem{Tprob, _A, _B, _C, _D, Val
```

This is the deflated Newton-Krylov Solver for computing a periodic orbit using a (Standard / Poincaré) Shooting method.

Arguments

Similar to `newton` except that `prob` is either a `Shooting` or a `PoincareShooting`.

Optional argument

`jacobian` Specify the choice of the linear algorithm, which must belong to `(BifurcationKit.AutoDiffMF(), BifurcationKit.MatrixFree(), BifurcationKit.AutoDiffDense(), BifurcationKit.AutoDiffDenseAnalytical(), BifurcationKit.FiniteDifferences(), BifurcationKit.FiniteDifferencesMF())`. This is used to select a way of inverting the jacobian dG

```
1. For `MatrixFree()`, matrix free jacobian, the jacobian is specified by the user in `prob`. This is
2. For `AutoDiffMF()`, we use Automatic Differentiation (AD) to compute the (matrix-free) derivative
3. For `AutodiffDense()`. Same as for `AutoDiffMF` but the jacobian is formed as a dense Matrix. You
```

GitHub

Output:

- solution::NonLinearSolution, see `NonLinearSolution`

```
newton(
    trap::Trapeze,
    orbitguess,
    options::NewtonPar;
    kwargs...
) -> NonLinearSolution{_A, Tprob, Tres} where {_A, Tprob<:(BifurcationKit.PeriodicOrbitFunctionalTrap
```

This is the Krylov-Newton Solver for computing a periodic orbit using a functional G based on finite differences and a Trapezoidal rule.

Arguments:

- `prob` a problem of type `Trapeze` encoding the functional G.
- `orbitguess` a guess for the periodic orbit. See `Trapeze` for more details.
- `par` parameters to be passed to the functional.
- `options` same as for the regular `newton` method.

Specify the choice of the jacobian (and linear algorithm), `jacobian` must belong to `(BifurcationKit.Dense(), BifurcationKit.AutoDiffDense(), BifurcationKit.FullLU(), BifurcationKit.FullMatrixFree(), BifurcationKit.BorderedLU(), BifurcationKit.BorderedMatrixFree(), BifurcationKit.FullSparseInplace(), BifurcationKit.BorderedSparseInplace(), BifurcationKit.AutoDiffMF())`. This is used to select a way of inverting the jacobian `dG` of the functional G.

- For `jacobian = FullLU()`, we use the default linear solver based on a sparse matrix representation of `dG`. This matrix is assembled at each newton iteration. This is the default algorithm. Can be used when the sparsity can change.
- For `jacobian = FullSparseInplace()`, this is the same as for `FullLU()` but the sparse matrix `dG` is updated inplace. This method allocates much less. In some cases, this is significantly faster than using `FullLU()`. Note that this method can only be used if the sparsity pattern of the jacobian is always the same.
- For `jacobian = Dense()`, same as above but the matrix `dG` is dense. It is also updated inplace. This option is useful to study ODE of small dimension.
- For `jacobian = AutoDiffDense()`, evaluate the jacobian using ForwardDiff
- For `jacobian = BorderedLU()`, we take advantage of the bordered shape of the linear solver and use a LU decomposition to invert `dG` using a bordered linear solver.
- For `jacobian = BorderedSparseInplace()`, this is the same as for `BorderedLU()` but the cyclic matrix `dG` is updated inplace. This method allocates much less. In some cases, this is significantly faster than using `:BorderedLU`. Note that this method can only be used if the sparsity pattern of the jacobian is always the same.
- For `jacobian = FullMatrixFree()`, a matrix free linear solver is used for `dG`: note that a preconditioner is very likely required here because of the cyclic shape of `dG` which affects negatively the convergence properties of GMRES.
- For `jacobian = BorderedMatrixFree()`, a matrix free linear solver is used but for `Jc` only (see docs): it means that `options.linsolver` is used to invert `Jc`. These two Matrix-Free options thus expose different part of the jacobian `dG` in order to use specific preconditioners. For example, an ILU preconditioner on `Jc` could remove the constraints in `dG` and lead to poor convergence. Of course, for these last two methods, a preconditioner is likely to be required.
- For `jacobian = AutoDiffMF()`, the evaluation map of the differential is derived using automatic differentiation. Thus, unlike the previous two cases, the user does not need to pass a Matrix-Free differential.

```
newton(
    trap::Trapeze,
    orbitguess,
    defOp::DeflationOperator{Tp, Tdot, T, vectype},
```

GitHub

This function is similar to `newton(probPO, orbitguess, options, jacobianPO; kwargs...)` except that it uses deflation in order to find periodic orbits different from the ones stored in `defOp`. We refer to the mentioned method for a full description of the arguments. The current method can be used in the vicinity of a Hopf bifurcation to prevent the Newton-Krylov algorithm from converging to the equilibrium point.

```
newton(
    coll::Collocation,
    orbitguess,
    options::NewtonPar;
    kwargs...
) -> NonLinearSolution{_A, Tprob, Tres} where {_A, Tprob<:(BifurcationKit.PeriodicOrbitFunctionalColl
```

This is the Newton solver for computing a periodic orbit using orthogonal collocation method. Note that the linear solver has to be apropriately set up in `options`.

Arguments

Similar to `newton` except that `prob` is a `Collocation`.

- `prob` a problem of type `<: Collocation` encoding the shooting functional G.
- `orbitguess` a guess for the periodic orbit.
- `options` same as for the regular `newton` method.

Optional argument

- `jacobian` Specify the choice of the linear algorithm, which must belong to `(AutoDiffDense(), )`. This is used to select a way of inverting the jacobian dG
  - For `AutoDiffDense()`. The jacobian is formed as a dense Matrix. You can use a direct solver or an iterative one using `options`. The jacobian is formed inplace.
  - For `DenseAnalytical()` Same as for `AutoDiffDense` but the jacobian is formed using a mix of AD and analytical formula.

```
newton(
    coll::Collocation,
    orbitguess,
    defOp::DeflationOperator,
    options::NewtonPar;
    kwargs...
) -> NonLinearSolution{_A, Tprob, Tres} where {_A, Tprob<:(DeflatedProblem{Tprob, Tp, _A, T, _B, Val{
```

This function is similar to `newton(::Collocation, orbitguess, options, jacobianPO; kwargs...)` except that it uses deflation in order to find periodic orbits different from the ones stored in `defOp`. We refer to the mentioned method for a full description of the arguments. The current method can be used in the vicinity of a Hopf bifurcation to prevent the Newton-Krylov algorithm from converging to the equilibrium point.

Continuation

`BifurcationKit.DotTheta` — Type

```
struct DotTheta{Tdot, Ta}
```

- ~~`dot::Any`: dot product used in pseudo-arclength constraint~~
- `apply!::Any`: Linear operator associated with dot product, i.e. dot(x, y) = <x, Ay>, where <,> is the standard dot product on R^N. You must provide an inplace function which evaluates A. For example `x -> rmul!(x, 1/length(x))`.

This parametric type allows to define a new dot product from the one saved in `dt::dot`. More precisely:

```
dt(u1, u2, p1::T, p2::T, theta::T) where {T <: Real}
```

computes, the weighted dot product $\langle (u_1, p_1), (u_2, p_2) \rangle_\theta = \theta \Re \langle u_1, u_2 \rangle + (1-\theta) p_1 p_2$ where $u_i \in \mathbb{R}^N$. The $\Re$ factor is put to ensure a real valued result despite possible complex valued arguments.

Info

This is used in the pseudo-arclength constraint with the dot product $\frac{1}{N}\langle u_1, u_2 \rangle, \quad u_i \in \mathbb{R}^N$

BifurcationKit.continuation — Function

```
continuation(
    prob::BifurcationKit.AbstractBifurcationProblem,
    alg::BifurcationKit.AbstractContinuationAlgorithm,
    contparams::ContinuationPar;
    linear_algo,
    bothside,
    kwargs...
) -> BifurcationKit.DCResult{Tprob, Tbr, Tit, Tsol, Talg} where {Tprob<:BifurcationKit.BVP.BVPBifProb
```

Compute the continuation curve associated to the functional `F` which is stored in the bifurcation problem `prob`. General information is available in Continuation methods: introduction.

Arguments:

- `prob::AbstractBifurcationFunction` a `::AbstractBifurcationProblem`, typically a `BifurcationProblem` which holds the vector field and its jacobian. We also refer to `BifFunction` for more details.
- `alg` continuation algorithm, for example `Natural(), PALC(), Multiple(),...`. See algos
- `contparams::ContinuationPar` parameters for continuation. See `ContinuationPar`

Optional Arguments:

- `plot = false` whether to plot the solution/branch/spectrum while computing the branch
- `bothside = true` compute the branches on the two sides of the initial parameter value `p0`, merge them and return it.
- `normC = norm` norm used in the nonlinear solves
- `filename` to save the computed branch during continuation. The identifier .jld2 will be appended to this filename. This requires `using JLD2`.
- `callback_newton` callback for newton iterations. See docs of `newton`. For example, it can be used to change the preconditioners. It can also be used to limit residuals and parameter steps, see `cbMaxNorm` and `cbMaxNormAndΔp`
- `finalise_solution = (z, tau, step, contResult; kwargs...) -> true` Function called at the end of each continuation step. Can be used to alter the continuation procedure (stop it by returning `false`), save personal data, plot... The notations are `z = BorderedArray(x, p)` where `x` (resp. `p`) is the current solution (resp. parameter value), `tau::BorderedArray` is the tangent at `z`, `step::Int` is the index of the current continuation step and `contResult` is the current branch but before saving the current `state` to it! For advanced use:
  - the state `state::ContState` of the continuation iterator is passed in `kwargs`. This can be used for testing whether this is called from bisection for locating bifurcation points / events: `in_bisection(state)` for example. This allows to escape some personal code in this case.

  Note that you can have a better control over the continuation procedure by using an iterator, see Iterator Interface.

`{0,1,2,3}`. In case `contparams.newton_options.verbose = false`, the following is valid (otherwise the newton iterations are shown). Each case prints more information than the previous one:

  - case 0: print nothing
  - case 1: print basic information about the continuation: used predictor, step size and parameter values
  - case 2: print newton iterations number, stability of solution, detected bifurcations / events
  - case 3: print information during bisection to locate bifurcations / events
- `linear_algo` set the linear solver for the continuation algorithm `alg`. For example, `PALC` needs a linear solver for an enlarged problem (size `n+1` instead of `n`) and one thus needs to tune the one passed in `contparams.newton_options.linsolver`. This is a convenient argument to thus change the `alg` linear solver and is used mostly internally. The proper way is to pass directly to `alg` the correct linear solver.
- `kind::AbstractContinuationKind` [Internal] flag to describe continuation kind (equilibrium, codim 2, ...). Default = `EquilibriumCont()`

Output:

- `contres::ContResult` composite type which contains the computed branch. See `ContResult` for more information.

Continuing the branch in the opposite direction

Just change the sign of `ds` in `ContinuationPar`.

Debug mode

Use debug mode to access more information about the progression of the continuation run, like iterative solvers convergence, problem update, ...

```
continuation(
    prob::BifurcationKit.AbstractBifurcationProblem,
    algdc::DefCont,
    contParams::ContinuationPar;
    verbosity,
    plot,
    linear_algo,
    dot_palc,
    callback_newton,
    filename,
    normC,
    kwcont...
) -> BifurcationKit.DCResult{Tprob, Tbr, Tit, Tsol, Talg} where {Tprob<:BifurcationKit.BVP.BVPBifProb
```

This function computes the set of curves of solutions `γ(s) = (x(s), p(s))` to the equation `F(x,p) = 0` based on the algorithm of **deflated continuation** as described in Farrell, Patrick E., Casper H. L. Beentjes, and Ásgeir Birkisson. “The Computation of Disconnected Bifurcation Diagrams.” ArXiv:1603.00809 [Math], March 2, 2016. http://arxiv.org/abs/1603.00809.

Depending on the options in `contParams`, it can locate the bifurcation points on each branch. Note that you can specify different predictors using `alg`.

Arguments:

- `prob::AbstractBifurcationProblem` bifurcation problem
- `alg::DefCont`, deflated continuation algorithm, see `DefCont`
- `contParams` parameters for continuation. See `ContinuationPar` for more information about the options

Optional Arguments:

- `plot = false` whether to plot the solution while computing,

passed the keyword argument `fromDeflatedNewton = true` to tell the user can it is not in the continuation part (regular newton) of the algorithm,

- `verbosity::Int` controls the amount of information printed during the continuation process. Must belong to `{0,⋯,5}`,
- `normC = norm` norm used in the Newton solves,
- `dot_palc = (x, y) -> dot(x, y) / length(x)`, dot product used to define the weighted dot product (resp. norm) $\|(x,p)\|_\theta^2$ in the constraint $N(x,p)$ (see online docs on PALC). This argument can be used to remove the factor `1/length(x)` for example in problems where the dimension of the state space changes (mesh adaptation, ...),

Outputs:

- `contres::DCResult` composite type which contains the computed branches. See `ContResult` for more information,

```
continuation(
    br::BifurcationKit.AbstractBranchResult,
    ind_bif,
    lens2::Union{typeof(identity), IndexLens, PropertyLens, ComposedFunction};
    ...
) -> Any
continuation(
    br::BifurcationKit.AbstractBranchResult,
    ind_bif,
    lens2::Union{typeof(identity), IndexLens, PropertyLens, ComposedFunction},
    options_cont::ContinuationPar;
    prob,
    start_with_eigen,
    detect_codim2_bifurcation,
    update_minaug_every_step,
    kwargs...
) -> Any
```

Codimension 2 continuation of Fold / Hopf points. This function turns an initial guess for a Fold / Hopf point into a curve of Fold / Hopf points based on a Minimally Augmented formulation. The arguments are as follows

- `br` results returned after a call to continuation
- `ind_bif` bifurcation index in `br`
- `lens2` second parameter used for the continuation, the first one is the one used to compute `br`, e.g. `getlens(br)`
- `options_cont = br.contparams` arguments to be passed to the regular continuation

Optional arguments:

- `linsolve_adjoint` solver for $(J+i\omega)^* \cdot sol = rhs$ or $J^t \cdot sol = rhs$
- `bdlinsolver` bordered linear solver for the constraint equation
- `bdlinsolver_adjoint` bordered linear solver for the constraint equation with top-left block $(J-i\omega)^*$ or J^t. Required in the linear solver for the Minimally Augmented Fold/Hopf functional. This option can be used to pass a dedicated linear solver for example with specific preconditioner.
- `update_minaug_every_step` update vectors `a, b` in Minimally Formulation every `update_minaug_every_step` steps
- `detect_codim2_bifurcation ∈ {0,1,2}` whether to detect Bogdanov-Takens, Bautin and Cusp. If equals `1` non precise detection is used. If equals `2`, a bisection method is used to locate the bifurcations. Default value = 2.
- `start_with_eigen = false` whether to start the Minimally Augmented problem with information from eigen elements. If `start_with_eigen = false`, then:
  - `a::Nothing` estimate of null vector of J (resp. J-iω) for Fold (resp. Hopf). If nothing is passed, a random vector is used. In case you do not rely on `AbstractArray`, you should probably pass this.
  - `b::Nothing` estimate of null vector of J^t (resp. $(J-i\omega)^*$) for Fold (resp. Hopf). If nothing is passed, a random vector is used. In case you do not rely on `AbstractArray`, you should probably pass this.
- `kwargs` keywords arguments to be passed to the regular continuation

GitHub

DDE problems

For ODE problems, it is more efficient to use the Matrix based Bordered Linear Solver passing the option `bdlinsolver = MatrixBLS()`

start_with_eigen

It is recommended that you use the option `start_with_eigen = true`

```
continuation(
    prob::BifurcationKit.AbstractBifurcationProblem,
    x0,
    par0,
    x1,
    p1::Real,
    alg,
    lens::Union{typeof(identity), IndexLens, PropertyLens, ComposedFunction},
    contParams::ContinuationPar;
    bothside,
    kwargs...
) -> ContResult
```

[Internal] This function is not meant to be called directly.

This function is the analog of `continuation` when the first two points on the branch are passed (instead of a single one). Hence `x0` is the first point on the branch (with pseudo arc length `s=0`) with parameter `par0` and `x1` is the second point with parameter `set(par0, lens, p1)`.

```
continuation(
    br::BifurcationKit.AbstractResult{Tkind<:Union{BifurcationKit.BoundaryValueProblemCont, Bifurcati
    ind_bif::Int64;
    ...
) -> Any
continuation(
    br::BifurcationKit.AbstractResult{Tkind<:Union{BifurcationKit.BoundaryValueProblemCont, Bifurcati
    ind_bif::Int64,
    options_cont::ContinuationPar;
    ...
) -> Branch
continuation(
    br::BifurcationKit.AbstractResult{Tkind<:Union{BifurcationKit.BoundaryValueProblemCont, Bifurcati
    ind_bif::Int64,
    options_cont::ContinuationPar,
    Teigvec::Type{𝒯eigvec};
    alg,
    δp,
    ampfactor,
    use_normal_form,
    bls,
    bls_block,
    nev,
    scaleζ,
    autodiff,
    usedeflation,
    verbosedeflation,
    max_iter_deflation,
    perturb,
    plot_solution,
    tol_fold,
    kwargs_deflated_newton,
```

GitHub

Automatic branch switching at branch points based on a computation of the normal form. More information is provided in Branch switching. An example of use is provided in 2d generalized Bratu–Gelfand problem.

Arguments

- `br`: branch result from a call to `continuation` containing the bifurcation point
- `ind_bif`: index of the bifurcation point in `br` from which to branch
- `options_cont`: continuation parameters for the new branch

Optional arguments

- `alg = getalg(br)` continuation algorithm to be used, default value: `getalg(br)`
- `δp` used to specify a specific value for the parameter on the bifurcated branch which is otherwise determined by `options_cont.ds`. This allows to use a step larger than `options_cont.dsmax`.
- `ampfactor = 1` factor to alter the amplitude of the bifurcated solution. Useful to magnify the bifurcated solution when the bifurcated branch is very steep. Can also be used to select the upper/lower branch in Pitchfork bifurcations. See also `use_normal_form` below.
- `use_normal_form = true`. If `use_normal_form = true`, the normal form is computed as well as its predictor and a guess is automatically formed. If `use_normal_form = false`, the parameter value `p = p₀ + δp` and the guess `x = x₀ + ampfactor .* e` (where `e` is a vector of the kernel) are used as initial guess. This is useful in case automatic branch switching does not work.
- `nev` number of eigenvalues to be computed to get the right eigenvector
- `scaleζ = norm` pass a norm to normalize vectors during normal form computation
- `plot_solution` change plot solution method in the problem `getprob(br)`
- `usedeflation = false` whether to use nonlinear deflation (see Deflated problems) to help finding the guess on the bifurcated
- `verbosedeflation` print deflated newton iterations
- `max_iter_deflation` number of newton steps in deflated newton
- `perturb = identity` which perturbation function to use during deflated newton
- `Teigvec = _getvectortype(br)` type of the eigenvector. Useful when `br` was loaded from a file and this information was lost
- `kwargs` optional arguments to be passed to `continuation`, the regular `continuation` one and to `get_normal_form`.

Advanced use

In the case of a very large model and use of special hardware (GPU, cluster), we suggest to decouple the computation of the reduced equation, the predictor and the bifurcated branches. Have a look at `methods(BifurcationKit.multicontinuation)` to see how to call these versions. These methods has been tested on GPU with very high memory pressure.

```
continuation(
    discPO::BifurcationKit.AbstractShootingDiscretization,
    orbitguess,
    alg::BifurcationKit.AbstractContinuationAlgorithm,
    contParams::ContinuationPar,
    linear_algo::BifurcationKit.AbstractBorderedLinearSolver;
    δ,
    eigsolver,
    record_from_solution,
    plot_solution,
    kwargs...
) -> Any
```

This is the continuation method for computing a periodic orbit using a (Standard / Poincaré) Shooting method.

Similar to continuation except that prob is either a Shooting or a PoincareShooting. By default, it prints the period of the periodic orbit.

GitHub

Optional arguments

- `eigsolver` specify an eigen solver for the computation of the Floquet exponents, defaults to `FloquetQaD`

`jacobian` Specify the choice of the linear algorithm, which must belong to `(BifurcationKit.AutoDiffMF(), BifurcationKit.MatrixFree(), BifurcationKit.AutoDiffDense(), BifurcationKit.AutoDiffDenseAnalytical(), BifurcationKit.FiniteDifferences(), BifurcationKit.FiniteDifferencesMF())`. This is used to select a way of inverting the jacobian dG

```
1. For `MatrixFree()`, matrix free jacobian, the jacobian is specified by the user in `prob`. This is
2. For `AutoDiffMF()`, we use Automatic Differentiation (AD) to compute the (matrix-free) derivative
3. For `AutodiffDense()`. Same as for `AutoDiffMF` but the jacobian is formed as a dense Matrix. You
4. For `FiniteDifferences()`, same as for `AutoDiffDense` but we use Finite Differences to compute th
5. For `AutoDiffDenseAnalytical()`. Same as for `AutoDiffDense` but the jacobian is formed using a mi
6. For `FiniteDifferencesMF()`, use Finite Differences to compute the matrix-free jacobian of `x -> p
```

```
continuation(
    disc::BifurcationKit.AbstractBoundaryValueDiscretization,
    orbitguess,
    alg::BifurcationKit.AbstractContinuationAlgorithm,
    _contParams::ContinuationPar;
    linear_algo,
    kwargs...
) -> Any
```

This is the continuation routine for computing a periodic orbit.

Arguments

Similar to `continuation` except that `prob::AbstractBoundaryValueDiscretization`.

Optional argument

- `linear_algo::AbstractBorderedLinearSolver`

`jacobian` Specify the choice of the linear algorithm, which must belong to `(BifurcationKit.AutoDiffMF(), BifurcationKit.MatrixFree(), BifurcationKit.AutoDiffDense(), BifurcationKit.AutoDiffDenseAnalytical(), BifurcationKit.FiniteDifferences(), BifurcationKit.FiniteDifferencesMF())`. This is used to select a way of inverting the jacobian dG

```
1. For `MatrixFree()`, matrix free jacobian, the jacobian is specified by the user in `prob`. This is
2. For `AutoDiffMF()`, we use Automatic Differentiation (AD) to compute the (matrix-free) derivative
3. For `AutodiffDense()`. Same as for `AutoDiffMF` but the jacobian is formed as a dense Matrix. You
4. For `FiniteDifferences()`, same as for `AutoDiffDense` but we use Finite Differences to compute th
5. For `AutoDiffDenseAnalytical()`. Same as for `AutoDiffDense` but the jacobian is formed using a mi
6. For `FiniteDifferencesMF()`, use Finite Differences to compute the matrix-free jacobian of `x -> p
```

```
continuation(
    br::BifurcationKit.AbstractBranchResult,
    ind_bif::Int64,
    _contParams::ContinuationPar,
    disc::BifurcationKit.AbstractBoundaryValueDiscretization;
    bif_prob,
    detailed,
    use_normal_form,
    autodiff_nf,
```

```
) -> Branch
```

GitHub

Perform automatic branch switching from a Hopf bifurcation point labelled `ind_bif` in the list of the bifurcated points of a previously computed branch `br::ContResult`. It first computes a Hopf normal form.

Arguments

- `br` branch result from a call to `continuation`
- `ind_hopf` index of the bifurcation point in `br`
- `contParams` parameters for the call to `continuation`
- `disc` discretization used to specify the way toc compute the periodic orbit. It can be `Trapeze`, `Collocation`, `Shooting` or `PoincareShooting` .

Optional arguments

- `alg = getalg(br)` continuation algorithm
- `δp` used to specify the guess for the parameter on the bifurcated branch which otherwise defaults to `contParams.ds`. This allows to use an initial step larger than `contParams.dsmax`.
- `ampfactor = 1` multiplicative factor to alter the amplitude of the bifurcated solution. Useful to magnify the bifurcated solution when the bifurcated branch is very steep.
- `use_normal_form = true` whether to use the normal form in order to compute the predictor. When `false`, `ampfactor` and `δp` are used to make a predictor based on the bifurcating eigenvector. Setting `use_normal_form = false` can be useful when computing the normal form is not possible for example when higher order derivatives are not available.
- `usedeflation = true` whether to use nonlinear deflation (see Deflated problems) to help finding the guess on the bifurcated branch
- `nev` number of eigenvalues to be computed to get the right eigenvector
- `autodiff_nf = true` whether to use `autodiff` in `get_normal_form`. This can be used in case automatic differentiation is not working as intended.
- all `kwargs` from `continuation`

A modified version of `prob` is passed to `plot_solution` and `finalise_solution`.

> **Linear solver**
>
> You have to be careful about the options `contParams.newton_options.linsolver`. In the case of Matrix-Free solver, you have to pass the right number of unknowns `N * M + 1`. Note that the options for the preconditioner are not accessible yet.

```
continuation(
    br::BifurcationKit.AbstractResult{BifurcationKit.PeriodicOrbitCont, Tprob<:BifurcationKit.Abstrac
    ind_bif::Int64,
    _contParams::ContinuationPar;
    alg,
    δp,
    ampfactor,
    usedeflation,
    linear_algo,
    detailed,
    prm,
    use_normal_form,
    autodiff_nf,
    kwargs...
) -> Branch
```

Branch switching at a bifurcation point of periodic orbits (PO) specified by a `br::AbstractBranchResult`. The functional for computing the PO is `getprob(br)`. A deflated Newton-Krylov solver can be used to improve the branch switching

Arguments

- `br` branch of periodic orbits
- `ind_bif` index of the branch point
- `_contParams` continuation parameters, see `continuation`

Optional arguments

- `δp = _contParams.ds` used to specify a particular guess for the parameter in the branch which is otherwise determined by `contParams.ds`. This allows to use a step larger than `contParams.dsmax`.
- `ampfactor = 1` factor which alters the amplitude of the bifurcated solution. Useful to magnify the bifurcated solution when the bifurcated branch is very steep.
- `usedeflation = true` whether to use nonlinear deflation (see Deflated problems) to help finding the guess on the bifurcated branch
- `use_normal_form = true` if `false`, the predictor is based on the couple `δp, ampfactor`.

For normal form

- `detailed = false` whether to fully compute the normal form or a very simplified version.
- `autodiff_nf = true` whether to use `autodiff` in `get_normal_form`. This can be used in case automatic differentiation is not working as intented.

For continuation

- `linear_algo = BorderingBLS()`, same as for `continuation`
- `kwargs` keywords arguments used for a call to the regular `continuation` and the ones specific to periodic orbits (POs).

```
continuation(
    trap::Trapeze,
    orbitguess,
    alg::BifurcationKit.AbstractContinuationAlgorithm,
    _contParams::ContinuationPar;
    959,
    record_from_solution,
    linear_algo,
    kwargs...
) -> Any
```

This is the continuation routine for computing a periodic orbit using a functional G based on finite differences and a Trapezoidal rule.

Arguments

- `prob::Trapeze` encodes the functional G.
- `orbitguess` a guess for the periodic orbit. See `Trapeze` for more details.
- `alg` continuation algorithm.
- `contParams` same as for the regular `continuation` method.

Keyword arguments

- `linear_algo` same as in `continuation`

Specify the choice of the jacobian (and linear algorithm), `jacobian` must belong to `(BifurcationKit.Dense(), BifurcationKit.AutoDiffDense(), BifurcationKit.FullLU(), BifurcationKit.FullMatrixFree(), BifurcationKit.BorderedLU(), BifurcationKit.BorderedMatrixFree(), BifurcationKit.FullSparseInplace(), BifurcationKit.BorderedSparseInplace(), BifurcationKit.AutoDiffMF())`. This is used to select a way of inverting the jacobian `dG` of the functional G.

- For `jacobian = FullLU()`, we use the default linear solver based on a sparse matrix representation of `dG`. This matrix is assembled at each newton iteration. This is the default algorithm. Can be used when the sparsity can change.

can only be used if the sparsity pattern of the jacobian is always the same.

- For `jacobian = Dense()`, same as above but the matrix `dG` is dense. It is also updated inplace. This option is useful to study ODE of small dimension.
- For `jacobian = AutoDiffDense()`, evaluate the jacobian using ForwardDiff
- For `jacobian = BorderedLU()`, we take advantage of the bordered shape of the linear solver and use a LU decomposition to invert `dG` using a bordered linear solver.
- For `jacobian = BorderedSparseInplace()`, this is the same as for `BorderedLU()` but the cyclic matrix `dG` is updated inplace. This method allocates much less. In some cases, this is significantly faster than using `:BorderedLU`. Note that this method can only be used if the sparsity pattern of the jacobian is always the same.
- For `jacobian = FullMatrixFree()`, a matrix free linear solver is used for `dG`: note that a preconditioner is very likely required here because of the cyclic shape of `dG` which affects negatively the convergence properties of GMRES.
- For `jacobian = BorderedMatrixFree()`, a matrix free linear solver is used but for `Jc` only (see docs): it means that `options.linsolver` is used to invert `Jc`. These two Matrix-Free options thus expose different part of the jacobian `dG` in order to use specific preconditioners. For example, an ILU preconditioner on `Jc` could remove the constraints in `dG` and lead to poor convergence. Of course, for these last two methods, a preconditioner is likely to be required.
- For `jacobian = AutoDiffMF()`, the evaluation map of the differential is derived using automatic differentiation. Thus, unlike the previous two cases, the user does not need to pass a Matrix-Free differential.

Note that by default, the method prints the period of the periodic orbit as function of the parameter. This can be changed by providing your `record_from_solution` argument.

```
continuation(
    coll::Collocation,
    orbitguess,
    alg::BifurcationKit.AbstractContinuationAlgorithm,
    _contParams::ContinuationPar,
    linear_algo::BifurcationKit.AbstractBorderedLinearSolver;
    δ,
    eigsolver,
    record_from_solution,
    plot_solution,
    kwargs...
) -> Any
```

This is the continuation method for computing a periodic orbit using an orthogonal collocation method.

Arguments

Similar to `continuation` except that `prob` is a `Collocation`. By default, it prints the period of the periodic orbit.

Keywords arguments

- `eigsolver` specify an eigen solver for the computation of the Floquet exponents, defaults to `FloquetQaD`

Continuation algorithms

Tangents / predictors

`BifurcationKit.Natural` — Type

Natural continuation algorithm. The predictor is the constant predictor and the parameter is incremented by `ContinuationPar().ds` at each continuation step.

Constructor(s)

`Natural()`

```
Secant Tangent predictor
```

GitHub

BifurcationKit.Bordered — Type

```
Bordered Tangent predictor
```

BifurcationKit.Polynomial — Type

```
Polynomial Tangent predictor
```

Internal fields

- `n::Int64`: Order of the polynomial
- `k::Int64`: Length of the last solutions vector used for the polynomial fit
- `A::Matrix{T} where T<:Real`: Matrix for the interpolation
- `tangent::BifurcationKit.AbstractTangentComputation`: Algo for tangent when polynomial predictor is not possible
- `solutions::DataStructures.CircularBuffer`: Vector of solutions
- `parameters::DataStructures.CircularBuffer{T} where T<:Real`: Vector of parameters
- `arclengths::DataStructures.CircularBuffer{T} where T<:Real`: Vector of arclengths
- `coeffsSol::Vector`: Coefficients for the polynomials for the solution
- `coeffsPar::Vector{T} where T<:Real`: Coefficients for the polynomials for the parameter
- `update::Bool`: Update the predictor by adding the last point (x, p)? This can be disabled in order to just use the polynomial prediction. It is useful when the predictor is called mutiple times during bifurcation detection using bisection.

Constructor(s)

```
Polynomial(pred, n, k, v0)

Polynomial(n, k, v0)
```

- `n` order of the polynomial
- `k` length of the last solutions vector used for the polynomial fit
- `v0` example of solution to be stored. It is only used to get the `eltype` of the tangent.

Can be used like

```
PALC(tangent = Polynomial(Bordered(), 2, 6, rand(1)))
```

BifurcationKit.Multiple — Type

```
Multiple Tangent continuation algorithm.
```

The predictor is designed [Uecker2014] to avoid spurious branch switching and pass singular points especially in PDE where branch point density can be quite high. It is called `pmcont` in `pde2path`.

Internal fields

- `alg::PALC`: Tangent predictor used. Default: PALC()
- `τ::Any`: Save the current tangent.
- `α::Real`: Damping in Newton iterations, 0 < α < 1.

- `pmimax::Int64`: Index for lookup in residual history. Default: 1
- `imax::Int64`: Maximum index for lookup in residual history. Default: 4
- `dsfact::Real`: Factor to increase ds upon successful step. Default: 1.5

Constructor(s)

```
Multiple(alg, x0, α, n)

Multiple(pred, x0, α, n)

Multiple(x0, α, n)
```

Algorithms

BifurcationKit.PALC — Type

```
struct PALC{Ttang<:BifurcationKit.AbstractTangentComputation, Tbls<:BifurcationKit.AbstractLinearSolv
```

Pseudo-arclength continuation algorithm.

Additional information is available on the website.

Internal fields

- `tangent::BifurcationKit.AbstractTangentComputation`: Tangent predictor, must be a subtype of `AbstractTangentComputation`. For example `Secant()` or `Bordered()`, Default: Secant()
- `θ::Any`: `θ` is a parameter in the arclength constraint. It is very **important** to tune it. It should be tuned for the continuation to work properly especially in the case of large problems where the < x - x0, *dx*0 > component in the constraint equation might be favoured too much. Also, large thetas favour p as the corresponding term in N involves the term 1-theta. Default: 0.5
- `_bothside::Bool`: [internal], Default: false
- `bls::BifurcationKit.AbstractLinearSolver`: Bordered linear solver used to invert the jacobian of the bordered problem during newton iterations. It is also used to compute the tangent for the predictor `Bordered()`, Default: MatrixBLS()
- `dotθ::Any`: `dotθ = DotTheta()`, this sets up a dot product `(x, y) -> dot(x, y) / length(x)` used to define the weighted dot product (resp. norm) $\|(x,p)\|_\theta^2$ in the constraint $N(x,p)$ (see online docs on PALC). This argument can be used to remove the factor `1/length(x)` for example in problems where the dimension of the state space changes (mesh adaptation, ...) or when a specific (FEM) dot product is provided. Default: DotTheta()

BifurcationKit.AutoSwitch — Type

```
struct AutoSwitch{Talg, T} <: BifurcationKit.AbstractContinuationAlgorithm
```

Continuation algorithm which switches automatically between Natural continuation and PALC (or other if specified) depending on the stiffness of the branch being continued. The formula for switching is:

```
(1-θ)*abs(τ.p) > tol_param
```

Internal fields

- `alg::Any`: Continuation algorithm to switch to when Natural is discarded. Typically `PALC()`
- `tol_param::Any`: tolerance for switching to PALC(), default value = 1//2

Constructor(s)

GitHub

BifurcationKit.MoorePenrose — Type

```
struct MoorePenrose{T, Tls<:BifurcationKit.AbstractLinearSolver} <: BifurcationKit.AbstractContinuati
```

Moore-Penrose continuation algorithm.

Additional information is available on the website.

Constructors

```
alg = MoorePenrose()
```

```
alg = MoorePenrose(tangent = PALC())
```

Internal fields

- `tangent::Any`: Tangent predictor, for example `PALC()`.
- `method::MoorePenroseLS`: Moore Penrose linear solver. Can be BifurcationKit.direct, BifurcationKit.plnv or BifurcationKit.iterative.
- `ls::BifurcationKit.AbstractLinearSolver`: (Bordered) linear solver.

Constructor(s)

```
MoorePenrose(;tangent = PALC(), method = direct, ls = nothing)
```

Missing docstring.

Missing docstring for `AsymptoticNumericalMethod.ANM`. Check Documenter's build log for details.

BifurcationKit.DefCont — Type

```
struct DefCont{Tdo, Talg, Tps, Tas, Tud, Tk} <: BifurcationKit.AbstractContinuationAlgorithm
```

Structure which holds the parameters specific to Deflated continuation.

Internal fields

- `deflation_operator::Any`: Deflation operator, `::DeflationOperator` Default: nothing
- `alg::Any`: Used as a predictor, `::AbstractContinuationAlgorithm`. For example `PALC()`, `Natural()`,... Default: PALC()
- `max_branches::Int64`: maximum number of (active) branches to be computed Default: 100
- `seek_every_step::Int64`: whether to seek new (deflated) solution at every step Default: 1
- `max_iter_defop::Int64`: maximum number of deflated Newton iterations Default: 5
- `perturb_solution::Any`: perturb function Default: _perturbSolution
- `accept_solution::Any`: accept (solution) function Default: _acceptSolution
- `update_deflation_op::Any`: function to update the deflation operator, ie pushing new solutions Default: _updateDeflationOp
- `jacobian::Any`: jacobian for deflated newton. Can be `DeflatedProblemCustomLS()`, or `Val(:autodiff)`, `Val(:fullIterative)` Default: DeflatedProblemCustomLS()

Events

BifurcationKit.DiscreteEvent — Type

Structure to pass a DiscreteEvent function to the continuation algorithm. A discrete call back returns a discrete value and we seek when it changes.

Internal fields

- `nb::Int64`: number of events, ie the length of the result returned by the callback function
- `condition::Any`: `= (iter, state) -> NTuple{nb, Int64}` callback function which at each continuation state, returns a tuple. For example, to detect a value change.
- `computeEigenElements::Bool`: whether the event requires to compute eigen elements
- `labels::Any`: Labels used to display information. For example `labels[1]` is used to qualify an event occurring in the first component. You can use `labels = ("hopf",)` or `labels = ("hopf", "fold")`. You must have `labels::Union{Nothing, NTuple{N, String}}`.
- `finaliser::Any`: Finaliser function
- `data::Any`: Place to store some personal data

BifurcationKit.ContinuousEvent — Type

```
struct ContinuousEvent{Tcb, Tl, T, Tf, Td} <: BifurcationKit.AbstractContinuousEvent
```

Structure to pass a ContinuousEvent function to the continuation algorithm. A continuous call back returns a **tuple/scalar** value and we seek its zeros.

Internal fields

- `nb::Int64`: number of events, i.e. the length of the result returned by the callback function
- `condition::Any`:, `(iter, state) -> NTuple{nb, T}` callback function which, at each continuation state, returns a tuple. For example, to detect crossing at 1.0 and at -2.0, you can pass `(iter, state) -> (getp(state)+2, getx(state)[1]-1)),`. Note that the type `T` should match the one of the parameter specified by the `::Lens` in `continuation`.
- `computeEigenElements::Bool`: whether the event requires to compute eigen elements
- `labels::Any`: Labels used to display information. For example `labels[1]` is used to qualify an event of the type `(0, 1.3213, 3.434)`. You can use `labels = ("hopf",)` or `labels = ("hopf", "fold")`. You must have `labels::Union{Nothing, NTuple{N, String}}`.
- `tol::Any`: Tolerance on event value to declare it as true event.
- `finaliser::Any`: Finaliser function
- `data::Any`: Place to store some personal data

BifurcationKit.SetOfEvents — Type

```
struct SetOfEvents{Tc<:Tuple, Td<:Tuple} <: BifurcationKit.AbstractEvent
```

Multiple events can be chained together to form a `SetOfEvents`. A `SetOfEvents` is constructed by passing to the constructor `ContinuousEvent`, `DiscreteEvent` or other `SetOfEvents` instances:

```
SetOfEvents(cb1, cb2, cb3)
```

Example

```
BifurcationKit.SetOfEvents(BK.FoldDetectCB, BK.BifDetectCB)
```

You can pass as many events as you like.

Internal fields

BifurcationKit.PairOfEvents — Type

```
struct PairOfEvents{Tc<:BifurcationKit.AbstractContinuousEvent, Td<:BifurcationKit.AbstractDiscreteEv
```

Structure to pass a PairOfEvents function to the continuation algorithm. It is composed of a pair ContinuousEvent / DiscreteEvent. A `PairOfEvents` is constructed by passing to the constructor a `ContinuousEvent` and a `DiscreteEvent`:

```
PairOfEvents(contEvent, discreteEvent)
```

Internal fields

- `eventC::BifurcationKit.AbstractContinuousEvent`: Continuous event
- `eventD::BifurcationKit.AbstractDiscreteEvent`: Discrete event

Branch switching (branch point)

BifurcationKit.continuation — Method

```
continuation(
    br::BifurcationKit.AbstractResult{Tkind<:Union{BifurcationKit.BoundaryValueProblemCont, Bifurcati
    ind_bif::Int64;
    ...
) -> Any
continuation(
    br::BifurcationKit.AbstractResult{Tkind<:Union{BifurcationKit.BoundaryValueProblemCont, Bifurcati
    ind_bif::Int64,
    options_cont::ContinuationPar;
    ...
) -> Branch
continuation(
    br::BifurcationKit.AbstractResult{Tkind<:Union{BifurcationKit.BoundaryValueProblemCont, Bifurcati
    ind_bif::Int64,
    options_cont::ContinuationPar,
    Teigvec::Type{𝒯eigvec};
    alg,
    δp,
    ampfactor,
    use_normal_form,
    bls,
    bls_block,
    nev,
    scaleζ,
    autodiff,
    usedeflation,
    verbosedeflation,
    max_iter_deflation,
    perturb,
    plot_solution,
    tol_fold,
    kwargs_deflated_newton,
    kwargs...
) -> Any
```

Automatic branch switching at branch points based on a computation of the normal form. More information is provided in Branch switching. An example of use is provided in 2d generalized Bratu–Gelfand problem.

- `br`: branch result from a call to `continuation` containing the bifurcation point

GitHub

- `ind_bif`: index of the bifurcation point in `br` from which to branch
- `options_cont`: continuation parameters for the new branch

Optional arguments

- `alg = getalg(br)` continuation algorithm to be used, default value: `getalg(br)`
- `δp` used to specify a specific value for the parameter on the bifurcated branch which is otherwise determined by `options_cont.ds`. This allows to use a step larger than `options_cont.dsmax`.
- `ampfactor = 1` factor to alter the amplitude of the bifurcated solution. Useful to magnify the bifurcated solution when the bifurcated branch is very steep. Can also be used to select the upper/lower branch in Pitchfork bifurcations. See also `use_normal_form` below.
- `use_normal_form = true`. If `use_normal_form = true`, the normal form is computed as well as its predictor and a guess is automatically formed. If `use_normal_form = false`, the parameter value `p = p₀ + δp` and the guess `x = x₀ + ampfactor .* e` (where `e` is a vector of the kernel) are used as initial guess. This is useful in case automatic branch switching does not work.
- `nev` number of eigenvalues to be computed to get the right eigenvector
- `scaleζ = norm` pass a norm to normalize vectors during normal form computation
- `plot_solution` change plot solution method in the problem `getprob(br)`
- `usedeflation = false` whether to use nonlinear deflation (see Deflated problems) to help finding the guess on the bifurcated
- `verbosedeflation` print deflated newton iterations
- `max_iter_deflation` number of newton steps in deflated newton
- `perturb = identity` which perturbation function to use during deflated newton
- `Teigvec = _getvectortype(br)` type of the eigenvector. Useful when `br` was loaded from a file and this information was lost
- `kwargs` optional arguments to be passed to `continuation`, the regular `continuation` one and to `get_normal_form`.

Advanced use

In the case of a very large model and use of special hardware (GPU, cluster), we suggest to decouple the computation of the reduced equation, the predictor and the bifurcated branches. Have a look at `methods(BifurcationKit.multicontinuation)` to see how to call these versions. These methods has been tested on GPU with very high memory pressure.

`BifurcationKit.multicontinuation` — Function

```
multicontinuation(
    br::BifurcationKit.AbstractBranchResult,
    ind_bif::Int64;
    ...
) -> Union{Nothing, Vector}
multicontinuation(
    br::BifurcationKit.AbstractBranchResult,
    ind_bif::Int64,
    options_cont::ContinuationPar;
    ...
) -> Union{Nothing, Vector}
multicontinuation(
    br::BifurcationKit.AbstractBranchResult,
    ind_bif::Int64,
    options_cont::ContinuationPar,
    Teigvec::Type{𝒯eigvec};
    δp,
    ampfactor,
    nev,
    ζs,
```

GitHub

```
    verbosedeflation,
    plot_solution,
    kwargs...
) -> Union{Nothing, Vector}
```

Automatic branch switching at branch points based on a computation of the normal form. More information is provided in Branch switching. An example of use is provided in 2d generalized Bratu–Gelfand problem.

Arguments

- `br` branch result from a call to `continuation`
- `ind_bif` index of the bifurcation point in `br` from which you want to branch from
- `options_cont` options for the call to `continuation`

Optional arguments

- `alg = getalg(br)` continuation algorithm to be used, default value: `getalg(br)`
- `δp` used to specify a particular guess for the parameter on the bifurcated branch which is otherwise determined by `options_cont.ds`. This allows to use a step larger than `options_cont.dsmax`.
- `ampfactor = 1` factor which alters the amplitude of the bifurcated solution. Useful to magnify the bifurcated solution when the bifurcated branch is very steep.
- `nev` number of eigenvalues to be computed to get the right eigenvector
- `verbosedeflation = true` whether to display the nonlinear deflation iterations (see Deflated problems) to help finding the guess on the bifurcated branch
- `scaleζ` norm used to normalize eigenbasis when computing the reduced equation
- `Teigvec` type of the eigenvector. Useful when `br` was loaded from a file and this information was lost
- `ζs` basis of the kernel
- `perturb_guess = identity` perturb the guess from the predictor just before the deflated-newton correction
- `kwargs` optional arguments to be passed to `continuation`, the regular `continuation` one.

Advanced use

In the case of a very large model and use of special hardware (GPU, cluster), we suggest to discouple the computation of the reduced equation, the predictor and the bifurcated branches. Have a look at `methods(BifurcationKit.multicontinuation)` to see how to call these versions. These methods has been tested on GPU with very high memory pressure.

```
multicontinuation(
    br::BifurcationKit.AbstractBranchResult,
    bpnf::BifurcationKit.NdBranchPoint,
    defOpm::DeflationOperator,
    defOpp::DeflationOperator;
    ...
) -> Vector
multicontinuation(
    br::BifurcationKit.AbstractBranchResult,
    bpnf::BifurcationKit.NdBranchPoint,
    defOpm::DeflationOperator,
    defOpp::DeflationOperator,
    options_cont::ContinuationPar;
    alg,
    δp,
    verbosedeflation,
    max_iter_deflation,
    plot_solution,
    Teigvec,
    kwargs...
) -> Vector
```

Arguments

- `br` branch result from a call to `continuation`
- `bpnf` normal form
- `defOpm::DeflationOperator, defOpp::DeflationOperator` to specify converged points on non-trivial branches before/after the bifurcation points. The points are located in `defOpm.roots` and `defOpp.roots`. Note that we only continue from the second points in the roots vectors, the first one is meant to be the trivial branch.

The rest is as the regular `multicontinuation` function.

Branch switching (Hopf point)

Missing docstring.

Missing docstring for `continuation(br::BifurcationKit.AbstractBranchResult, ind_bif::Int, _contParams::ContinuationPar, prob::BifurcationKit.AbstractPeriodicOrbitProblem ; kwargs...)`. Check Documenter's build log for details.

Bifurcation diagram

`BifurcationKit.bifurcationdiagram` — Function

```
bifurcationdiagram(
    prob::BifurcationKit.AbstractBifurcationProblem,
    alg::BifurcationKit.AbstractContinuationAlgorithm,
    level::Int64,
    options;
    linear_algo,
    kwargs...
) -> BifDiagNode{Tγ, Vector{BifDiagNode}} where Tγ<:BifurcationKit.AbstractBranchResult
```

Compute the bifurcation diagram associated with the problem `F(x, p) = 0` recursively.

Arguments

- `prob::AbstractBifurcationProblem` bifurcation problem
- `alg` continuation algorithm
- `level` maximum branching (or recursion) level for computing the bifurcation diagram
- `options = (x, p, level) -> contparams` this function allows to change the `continuation` options depending on the branching `level`. The argument `x, p` denotes the current solution to `F(x, p) = 0`.
- `kwargs` optional arguments. Look at `bifurcationdiagram!` for more details.

Simplified call:

We also provide the method

```
bifurcationdiagram(prob, br::ContResult, level::Int, options; kwargs...)
```

where `br` is a branch computed after a call to `continuation` from which we want to compute the bifurcating branches recursively.

`BifurcationKit.bifurcationdiagram!` — Function

```
bifurcationdiagram!(
    prob::BifurcationKit.AbstractBifurcationProblem,
    node::BifDiagNode,
```

```
    halfbranch,
    verbosediagram,
    kwargs...
) -> BifDiagNode
```

Similar to `bifurcationdiagram` but you pass a previously computed `node` from which you want to further compute the bifurcated branches. It is usually used with `node = get_branch(diagram, code)` from a previously computed bifurcation `diagram`.

Arguments

- `node::BifDiagNode` a node in the bifurcation diagram
- `maxlevel = 1` required maximal level of recursion.
- `options = (x, p, level; k...) -> contparams` this function allows to change the `continuation` options depending on the branching `level`. The argument `x, p` denotes the current solution to `F(x, p)=0`.

Optional arguments

- `code = "0"` code used to display iterations
- `usedeflation = false`
- `halfbranch = false` for Pitchfork / Transcritical bifurcations, compute only half of the branch. Can be useful when there are symmetries.
- `verbosediagram` verbose specific to bifurcation diagram. Print information about the branches as they are being computed.
- `kwargs` optional arguments as for `continuation` but also for the different versions listed in Continuation.

BifurcationKit.get_branch — Function

```
get_branch(diagram::BifDiagNode, code) -> BifDiagNode
```

Return the part of the diagram (bifurcation diagram) by recursively descending down the diagram using the `Int` valued tuple `code`. For example `get_branch(diagram, (1,2,3,))` returns `diagram.child[1].child[2].child[3]`.

BifurcationKit.get_branches_from_BP — Function

```
get_branches_from_BP(
    diagram::BifDiagNode,
    indbif::Int64
) -> Vector{BifDiagNode}
```

Return the part of the diagram corresponding to the indbif-th bifurcation point on the root branch.

Utils for periodic orbits

BifurcationKit.getperiod — Function

```
getperiod(
    ::BifurcationKit.AbstractBoundaryValueDiscretization,
    x
) -> Any
getperiod(
    ::BifurcationKit.AbstractBoundaryValueDiscretization,
    x,
    par
) -> Any
```

```
getperiod(prob::Trapeze, x, p) -> Any
```

GitHub

Compute the period of the periodic orbit associated to `x`.

```
getperiod(psh::PoincareShooting, x_bar, par) -> Any
```

Compute the period of the periodic orbit associated to `x_bar`.

BifurcationKit.SectionSS — Type

```
struct SectionSS{Tn} <: BifurcationKit.AbstractSection
```

This composite type (named for Section Standard Shooting) encodes a type of section implemented by a single hyperplane. It can be used in conjunction with `Shooting`. The hyperplane is defined by a point `center` and a `normal`.

Internal fields

- `normal::Any`: Normal to define hyperplane.
- `center::Any`: Representative point on hyperplane.

Constructor(s)

```
SectionSS(normals, centers)
```

BifurcationKit.SectionPS — Type

```
struct SectionPS{Tn, Tc, Tnb, Tcb, Tr} <: BifurcationKit.AbstractSection
```

This composite type (named for SectionPoincaréShooting) encodes a type of Poincaré sections implemented by hyperplanes. It can be used in conjunction with `PoincareShooting`. Each hyperplane is defined par a point (one example in `centers`) and a normal (one example in `normals`).

Internal fields

- `M::Int64`
- `normals::Any`
- `centers::Any`
- `indices::Vector{Int64}`
- `normals_bar::Any`
- `centers_bar::Any`
- `radius::Any`

Constructor(s)

```
SectionPS(normals, centers)
```

Misc.

BifurcationKit.PrecPartialSchurKrylovKit — Function

```
PrecPartialSchurKrylovKit(J, x0, nev, which = :LM; krylovdim = max(2nev, 20), verbosity = 0)
```

ones of `EigKrylovKit()`.

GitHub

`BifurcationKit.PrecPartialSchurArnoldiMethod` — Function

```
PrecPartialSchurArnoldiMethod(J, N, nev, which = LM(); tol = 1e-9, kwargs...)
```

Builds a preconditioner based on deflation of `nev` eigenvalues chosen according to `which`. A partial Schur decomposition is computed (Matrix-Free), using the package `ArnoldiMethod.jl`, from which a projection is built. See the package `ArnoldiMethod.jl` for how to pass the proper options.

`BifurcationKit.Flow` — Type

```
struct Flow{TF, Tf, Tts, Tff, Td, Tad, Tse, Tprob, TprobMono, Tfs, Tcb, Tδ} <: BifurcationKit.Abstrac
```

Structure to encode the flow associated to a Cauchy problem `dx/dt = F(x, p)`.

Internal fields

- `F::Any`: The vector field `(x, p) -> F(x, p)` associated to a Cauchy problem. Used for the differential of the shooting problem. Default: nothing
- `flow::Any`: The flow (or semigroup) `(x, p, t) -> flow(x, p, t)` associated to the Cauchy problem. Only the last time point must be returned in the form (u = ...) Default: nothing
- `flowTimeSol::Any`: Flow which returns the tuple (t, u(t)). Optional, mainly used for plotting on the user side. Default: nothing
- `flowFull::Any`: [Optional] The flow (or semigroup) associated to the Cauchy problem `(x, p, t) -> flow(x, p, t)`. The whole solution on the time interval [0,t] must be returned. It is not strictly necessary to provide this, it is mainly used for plotting on the user side. Please use `nothing` as default. Default: nothing
- `jvp::Any`: The differential `dflow` of the flow *w.r.t.* `x`, `(x, p, dx, t) -> dflow(x, p, dx, t)`. One important thing is that we require `dflow(x, dx, t)` to return a Named Tuple: `(t = t, u = flow(x, p, t), du = dflow(x, p, dx, t))`, the last component being the value of the derivative of the flow. Default: nothing
- `vjp::Any`: The adjoint differential `vjpflow` of the flow *w.r.t.* `x`, `(x, p, dx, t) -> vjpflow(x, p, dx, t)`. One important thing is that we require `vjpflow(x, p, dx, t)` to return a Named Tuple: `(t = t, u = flow(x, p, t), du = vjpflow(x, p, dx, t))`, the last component being the value of the derivative of the flow. Default: nothing
- `jvpSerial::Any`: [Optional] Serial version of dflow. Used internally when using parallel multiple shooting. Please use `nothing` as default. Default: nothing
- `prob::Any`: [Internal] store the ODEProblem associated to the flow of the Cauchy problem Default: nothing
- `probMono::Any`: [Internal] store the ODEProblem associated to the flow of the variational problem Default: nothing
- `flowSerial::Any`: [Internal] Serial version of the flow Default: nothing
- `callback::Any`: [Internal] Store possible callback Default: nothing
- `delta::Any`: [Internal] Default: 1.0e-8

Simplified constructor(s)

We provide a simple constructor where you only pass the vector field `F`, the flow `ϕ` and its differential `dϕ`:

```
fl = Flow(F, ϕ, dϕ)
```

Simplified constructors for DifferentialEquations.jl

These are some simple constructors for which you only have to pass a `prob::ODEProblem` or `prob::EnsembleProblem` (for parallel computation) from `DifferentialEquations.jl` and an ODE time stepper like `Tsit5()`. Hence, you can do for example

where `kwargs` is passed to `SciMLBase::solve`. If your vector field depends on parameters `p`, you can define a `Flow` using GitHub

```
fl = Flow(prob, Tsit5(); kwargs...)
```

Finally, you can pass two `ODEProblem` where the second one is used to compute the variational equation:

```
fl = Flow(prob1::ODEProblem, alg1, prob2::ODEProblem, alg2; kwargs...)
```

Missing docstring.

Missing docstring for `FloquetQaD`. Check Documenter's build log for details.

`BifurcationKit.get_normal_form` — Function

```
get_normal_form(
    prob::BifurcationKit.AbstractBifurcationProblem,
    br::BifurcationKit.AbstractBranchResult,
    id_bif::Int64;
    ...
) -> Any
get_normal_form(
    prob::BifurcationKit.AbstractBifurcationProblem,
    br::BifurcationKit.AbstractBranchResult,
    id_bif::Int64,
    Teigvec::Type{𝒯eigvec};
    nev,
    verbose,
    lens,
    detailed,
    autodiff,
    scaleζ,
    ζs,
    ζs_ad,
    bls,
    bls_adjoint,
    bls_block,
    start_with_eigen
) -> Any
```

Compute the reduced equation / normal form of the bifurcation point located at `br.specialpoint[ind_bif]`.

Arguments

- `prob::AbstractBifurcationProblem`
- `br` result from a call to `continuation`
- `ind_bif` index of the bifurcation point in `br.specialpoint`

Optional arguments

- `nev` number of eigenvalues used to compute the spectral projection. This number has to be adjusted when used with iterative methods.
- `verbose` whether to display information
- `ζs` list of vectors spanning the kernel of the jacobian at the bifurcation point. Useful for enforcing the kernel basis used for the normal form.
- `lens::Lens` specify which parameter to take the partial derivative ∂pF
- `scaleζ` function to normalize the kernel basis. Indeed, the kernel vectors are normalized using `norm`, the normal form coefficients can be super small and can imped its analysis. Using `scaleζ = norminf` can help sometimes.

- `detailed = Val(true)` whether to compute only a simplified normal form when only basic information is required. This can be useful is cases the computation is "long", for example for a Bogdanov-Takens point.
- `bls = MatrixBLS()` specify bordered linear solver. Needed to compute the reduced equation Taylor expansion of Branch/BT points. Indeed, it is required to solve `L•u = rhs` where `L` is the jacobian at the bifurcation point, `L` is thus singular and we rely on a bordered linear solver to solve this system.
- `bls_block = bls` specify bordered linear solver when the border has dimension > 1 (1 for `bls`). (see `bls` option above).

Available method(s)

You can directly call

```
get_normal_form(br, ind_bif ; kwargs...)
```

which is a shortcut for `get_normal_form(getprob(br), br, ind_bif ; kwargs...)`.

Once the normal form `nf` has been computed, you can call `predictor(nf, δp)` to obtain an estimate of the bifurcating branch.

References

[1] Golubitsky, Martin, and David G Schaeffer. Singularities and Groups in Bifurcation Theory. Springer-Verlag, 1985. http://books.google.com/books?id=rrg-AQAAIAAJ.

[2] Kielhöfer, Hansjörg. Bifurcation Theory: An Introduction with Applications to PDEs. Applied Mathematical Sciences 156. Springer, 2003. https://doi.org/10.1007/978-1-4614-0502-3.

```
get_normal_form(
    wrap::BifurcationKit.AbstractWrapperPeriodicOrbitProblem,
    br::BifurcationKit.AbstractResult{<:BifurcationKit.PeriodicOrbitCont},
    id_bif::Int64;
    ...
) -> Any
get_normal_form(
    wrap::BifurcationKit.AbstractWrapperPeriodicOrbitProblem,
    br::BifurcationKit.AbstractResult{<:BifurcationKit.PeriodicOrbitCont},
    id_bif::Int64,
    Teigvec::Type{𝒯eigvec};
    nev,
    verbose,
    ζs,
    lens,
    scaleζ,
    autodiff,
    δ,
    k...
) -> Any
```

Compute the normal form (NF) of bifurcations of periodic orbits. We detail the additional keyword arguments specific to periodic orbits.

Optional arguments

- `prm = true` compute the normal form using Poincaré return map (PRM). If false, use the looss normal form.
- `nev = length(eigenvalsfrombif(br, id_bif))`,
- `verbose = false`,
- `ζs = nothing`, pass the eigenvectors
- `lens = getlens(br)`,
- `Teigvec = _getvectortype(br)` type of the eigenvectors (can be useful for GPU)
- `scaleζ = norm`, scale the eigenvector

be useful is cases the computation is long.

GitHub

- `δ = getdelta(prob)` delta used for derivatives based on finite differences.

Notes

For collocation, the default method to compute the NF of Period-doubling and Neimark-Sacker bifurcations is Iooss' one [1].

References

[1] Iooss, "Global Characterization of the Normal Form for a Vector Field near a Closed Orbit.", 1988

« 🔄 Migration from old versions